# Assembly Preview Tab — Detailed Outline

# Assembly Preview Tab — Detailed Outline

## 1. Purpose

Build the **Preview** tab into a live in-app browser surface that lets the user open and inspect the assembly's built output as an actual web experience.

This tab should answer:

- What does this build look like when opened?
- Is the current build usable?
- Does it behave correctly across viewport sizes?
- Which run is this preview coming from?
- Can I refresh, relaunch, or open it outside the app?

## 2. Role of the Preview Tab

The tab should be positioned as the **runtime visual validation layer** of the assembly page.

**What it is**

- a live preview surface
- an embedded browser/iframe workspace
- a build-output validation screen
- a run-linked visual QA layer

**What it is not**

- not a file browser
- not a raw artifact viewer
- not a code editor
- not a template renderer panel
- not a document previewer unless the build itself is a document-style web output

## 3. Product Goal

Turn Preview into the place where the user can move from:

- **pipeline completed**
  to
- **I can now see and validate the real result**

This makes the tab the bridge between:

- **Build** = run/build process
- **Preview** = visible result of the build

---

# 4. Core User Outcomes

## 4.1 Immediate Visual Access

User opens the Preview tab and sees the latest available built output without hunting for links or artifacts.

## 4.2 Run-Aware Context

User knows exactly which run the preview belongs to.

## 4.3 Responsive Validation

User can inspect the build in desktop, tablet, and mobile viewport modes.

## 4.4 Fast Recovery

If preview is unavailable, the user gets a clear reason and a path forward.

## 4.5 External Launch

User can open the preview in a full browser tab when they need more room or native behavior.

---

# 5. Main Sections of the Tab

## 5.1 Preview Control Bar

The top control strip should act like a browser header for the build preview.

### Contents

- preview title
- current preview URL or route
- source run badge
- build status badge
- refresh action

- open in new tab action
- viewport/device toggle
- optional route selector
- optional reload-on-new-build toggle

**Goals**

- make the preview feel live
- make state and source obvious
- give the user fast control without leaving the tab

---

## 5.2 Main Preview Surface

This is the central viewing area where the built application or page is rendered.

**Behavior**

- render the build in an iframe or embedded webview
- occupy the majority of the tab
- support loading, ready, failed, and unavailable states
- adapt frame width based on selected device mode

**Modes**

- full desktop
- tablet frame
- mobile frame

**Optional enhancements later**

- zoom controls
- safe-area simulation
- orientation toggle
- dark/light forced preview modes

---

## 5.3 Preview Status / Metadata Panel

A side panel or bottom strip should expose operational context for the loaded preview.

**Contents**

- run id

- run status
- build status
- preview readiness
- generated timestamp
- last refreshed timestamp
- target environment
- entry route
- preview host/source
- failure summary if applicable

**Why it matters**

The user should never wonder:

- where this preview came from
- whether it is stale
- whether it reflects the latest build

---

### 5.4 Empty / Transitional State Area

When there is no valid preview to show, the tab must still be useful.

**Possible states**

- no run exists
- run exists but no build
- build exists but preview is not published
- build is in progress
- build failed
- preview URL exists but cannot load
- preview expired or was cleaned up

**Required response**

Every state should explain:

- what happened
- what the user can do next

---

## 6. Required UX States

## 6.1 No Assembly Run Yet

Show a clear empty state when nothing has been built.

**Message intent**

- this assembly has no preview yet
- run the pipeline/build to generate one

**Actions**

- go to Build tab
- start pipeline if permitted

---

## 6.2 Pipeline Running / Build in Progress

If the assembly is actively running or building, the tab should feel live.

**Display**

- spinner/loading panel
- current status text
- latest run id
- elapsed time if available
- optional polling indicator

**Actions**

- refresh status
- open Build tab
- inspect current run

---

## 6.3 Build Completed, Preview Preparing

A distinct state for when the build succeeded but the preview endpoint is not ready yet.

**Display**

- build complete, preview initializing
- waiting for deploy/publish/serve step
- last updated timestamp

**Actions**

- refresh
- retry preview resolution
- open Build tab

---

## 6.4 Preview Ready

The ideal steady-state experience.

**Display**

- embedded preview surface
- route/URL visible
- viewport toggle
- run badge
- open external action

**Actions**

- reload iframe
- switch viewport
- open in browser
- copy link
- inspect another run if supported later

---

## 6.5 Build Failed

This state should be concise and operational.

**Display**

- failure banner
- short reason
- related run id
- timestamp
- optionally stage or build step that failed

**Actions**

- go to Build tab
- inspect logs
- rerun build
- refresh preview state

---

### 6.6 Preview Load Failure

This is separate from build failure.

The build may have succeeded, but the preview URL may still fail to load due to host issues, CORS, expired session, or preview service instability.

**Display**

- preview could not be loaded
- URL/endpoint status
- last successful preview time if known

**Actions**

- reload
- reopen
- regenerate preview
- open externally

---

## 7. Functional Requirements

### 7.1 Resolve Current Preview Source

The tab must determine which run and build should power the preview.

**Default logic**

- prefer latest successful buildable run
- if assembly has active run and preview is live, use that
- otherwise use most recent ready preview

**Rules**

- never silently show stale content without indicating source
- always show run origin

---

### 7.2 Load Preview URL

The tab must resolve a browser-safe URL or embeddable endpoint.

**Requirements**

- URL must be associated with a run/build
- URL must support iframe embedding or provide a fallback
- URL must be validated before display

---

## 7.3 Refresh / Reload

The user needs two kinds of refresh:

**State refresh**

Recheck whether a preview exists and whether it is ready.

**Frame refresh**

Reload the embedded preview itself without rebuilding the whole page state.

---

## 7.4 Open in New Tab

The user should be able to open the preview outside the app.

**Why**

- better interaction space
- easier debugging
- native browser behavior
- useful when iframe restrictions exist

---

## 7.5 Responsive Device Modes

Preview should support at least three visual viewport presets.

**Desktop**

- full-width workspace
- no frame chrome

**Tablet**

- constrained centered width
- medium viewport simulation

**Mobile**

- narrow portrait frame
- centered device shell feel

---

### 7.6 Polling / Live Status

If the page already refetches assembly status while running, Preview should align with that behavior rather than becoming static. The assembly page already uses polling while status is `running`, which makes Preview a natural fit for readiness polling as well.

assembly

**Poll when**

- assembly is running
- build is processing
- preview is pending
- refresh was requested manually

**Stop polling when**

- preview is ready
- build failed
- no previewable run exists

---

## 8. Information Architecture

### 8.1 Header Layer

Persistent control area across all preview states.

**Contains**

- title: Preview
- status badge
- run badge
- route/URL

- actions

---

### 8.2 Viewport Layer

Primary area where the embedded result appears.

**Contains**

- iframe/webview
- frame shell depending on device mode
- loader or error overlay when needed

---

### 8.3 Context Layer

Supplemental state and metadata.

**Contains**

- build origin
- timestamps
- environment
- entry path
- logs/error shortcuts

---

## 9. UI Controls Breakdown

### 9.1 Top-Right Actions

- Refresh state
- Reload preview
- Open in new tab
- Copy preview URL

### 9.2 View Mode Controls

- Desktop
- Tablet
- Mobile

### 9.3 Optional Route Controls

Later version:

- root route
- named pages
- common app routes detected from build manifest

### 9.4 Optional Build Controls

- Rebuild
- Relaunch preview
- Switch run

---

## 10. Preview Data Contract

### 10.1 Needed Frontend Data

The tab needs a dedicated preview data source rather than trying to infer everything from files.

**Suggested fields**

- status
- runId
- buildStatus
- previewUrl
- entryUrl
- updatedAt
- error
- environment
- expiresAt
- embeddable

### 10.2 Suggested API Contract

Example shape:

```
{
 status: "none" | "building" | "preparing" | "ready" | "failed" | "expired";
 runId: string | null;
 buildStatus: string | null;
 previewUrl: string | null;
```

```
 entryUrl: string | null;
 updatedAt: string | null;
 expiresAt?: string | null;
 embeddable?: boolean;
 error?: string | null;
}
```

---

# 11. Backend Support Outline

## 11.1 Read Preview Status

A route to fetch the current preview state for an assembly.

**Responsibility**

- determine latest preview-capable build
- resolve preview URL
- return state summary

## 11.2 Refresh Preview Resolution

A route to force a re-check of preview readiness.

## 11.3 Optional Preview Session Creation

If preview links are ephemeral, backend may need to issue or renew a session.

## 11.4 Optional Route Manifest

If the build exports known routes/pages, backend can return those for route switching.

---

# 12. Component Architecture

## 12.1 `AssemblyPreviewTab`

Top-level controller for preview behavior.

**Responsibilities**

- fetch preview state

- poll when needed
- manage current device mode
- manage reload/open actions
- render state-specific UI

---

## 12.2 `PreviewToolbar`

Browser-like command strip.

**Responsibilities**

- show URL and state
- expose actions
- switch device mode

---

## 12.3 `PreviewViewport`

Embeds and displays the live output.

**Responsibilities**

- render iframe
- apply viewport width presets
- show load overlays
- show frame failure state

---

## 12.4 `PreviewMetaPanel`

Displays source/build context.

**Responsibilities**

- run info
- timestamps
- status
- environment
- diagnostics shortcuts

---

### 12.5 `PreviewEmptyState`

Shared state component for unavailable preview cases.

**Responsibilities**

- show clear explanation
- present correct next action

---

# 13. Visual Design Direction

## 13.1 Overall Feel

This tab should feel more like:

- a browser lab
- a deployment inspection surface
- a polished runtime viewer

Not:

- a cards-only placeholder
- a document panel
- a utilities drawer

## 13.2 Tone

Because the page already uses a futuristic glass-panel style, Preview should feel like a “live viewport console.”

**Design cues**

- browser-like address strip
- status pills
- centered framed device modes
- subtle loading overlays
- strong empty/error states

---

# 14. Behavior Rules

### 14.1 Always Show Source Context

Even when preview is ready, always show:

- source run
- preview status
- last updated time

### 14.2 Never Pretend a Preview Exists

Do not show a fake frame or static mock when no preview URL is available.

### 14.3 Separate Build Failure from Preview Failure

These are different problems and should have different messages.

### 14.4 Default to Latest Valid Preview

But make that source visible so the user knows whether it is fresh.

### 14.5 Preserve User View Mode

If the user chooses mobile mode, keep it while refreshing or polling.

---

## 15. MVP Scope

### 15.1 Must Have

- real Preview tab component
- preview status fetch
- iframe/webview embed
- desktop/tablet/mobile modes
- refresh action
- open in new tab
- no-build / building / failed / ready states
- run/source metadata

### 15.2 Should Have

- polling while build is running
- clear load-failure handling
- copy preview link

- entry route display

### 15.3 Later

- route switcher
- preview history by run
- screenshot capture
- compare two runs
- console/network overlays
- device orientation toggle
- preview annotations

---

## 16. Integration With Existing Assembly Page

The current page already has:

- the `preview` tab in the tab set
- a tab render branch for `activeTab === "preview"`
- assembly-level data loading
- run-aware state on the page

So this work is mainly about replacing the existing placeholder panel with a real preview module, not inventing a new page or navigation flow.

assembly

assembly

### Integration plan

- keep `preview` in existing `TABS`
- replace placeholder content under `activeTab === "preview"`
- mount `AssemblyPreviewTab`
- pass assembly id, current run id, and pipeline/build state
- align refresh behavior with assembly query state

---

## 17. Success Criteria

The Preview tab is complete when:

- opening the tab shows the actual built output when available
- the user can tell which run the preview came from
- the user can inspect the build at multiple viewport sizes
- the user can refresh and open externally
- missing/failed states are explicit and actionable
- the tab feels like a live runtime surface, not a placeholder card

---

# 18. Recommended Build Order

### Phase 1 — Structure

- create `AssemblyPreviewTab`
- add toolbar shell
- add viewport shell
- add metadata area

### Phase 2 — Data Wiring

- connect preview status endpoint
- resolve preview URL
- wire ready/building/failed states

### Phase 3 — Runtime UX

- iframe loading
- reload action
- open external action
- viewport toggles

### Phase 4 — Hardening

- polling
- load-failure handling
- stale/expired preview handling
- embeddability fallback

### Phase 5 — Enhancements

- route selector
- session renewals
- run switching
- compare mode

# Assembly Preview Tab — Build Spec

# Assembly Preview Tab — Build Spec

## 1. Feature Name

**Assembly Preview Tab**

## 2. Objective

Replace the current placeholder Preview tab with a real **embedded web preview workspace** that lets the user view the assembly's built output inside the app as if it were opened in a browser.

This tab must let the user:

- see the current built result
- know which run produced it
- inspect it at multiple viewport sizes
- refresh the preview state
- open the preview in a full browser tab
- recover cleanly from missing, loading, or failed preview states

---

## 3. Current Baseline

The current assembly page already provides the structural pieces needed for this feature:

- `preview` is already part of the tab model and nav strip.
  assembly
- assembly data is already fetched through a page-level query keyed by assembly id.
  assembly
- the page already refetches while assembly status is `running`, which can be extended to preview readiness behavior.
  assembly
- the current Preview tab body is only a placeholder and is incorrectly framed around artifacts/documents instead of web output.
  assembly

So this is a **replacement of the tab body**, not a new page.

---

# 4. Product Definition

## 4.1 What Preview Is

A runtime-facing tab that embeds the assembly's build output in a browser-like viewing surface.

## 4.2 What Preview Is Not

It is not:

- an artifacts explorer
- a file renderer
- a raw document viewer
- a code inspection panel

Artifacts already covers file-oriented output, while Preview should cover the **actual rendered result**. The current placeholder text should be fully retired.

assembly

---

# 5. User Story Set

## 5.1 Primary User Story

As an operator viewing an assembly, I want to open the Preview tab and immediately see the latest build rendered in-place so I can validate the result without leaving the assembly page.

## 5.2 Supporting User Stories

As an operator, I want to know which run the preview belongs to so I can trust what I am seeing.

As an operator, I want desktop/tablet/mobile view modes so I can quickly check responsiveness.

As an operator, I want to open the preview in a new browser tab when I need full interaction space.

As an operator, I want clear feedback when a preview does not exist, is still preparing, or failed to load.

---

# 6. Functional Scope

## 6.1 In Scope for MVP

- real Preview tab component
- preview-status data fetch
- preview URL resolution
- embedded iframe/webview surface
- desktop/tablet/mobile viewport modes
- refresh preview state
- reload preview frame
- open preview in new tab
- ready / no preview / building / failed / load-error states
- preview metadata panel with run/source context

## 6.2 Out of Scope for MVP

- console inspector
- network inspector
- side-by-side preview diff
- screenshot capture
- route crawling
- DOM overlays
- annotation/commenting
- run-to-run compare mode

---

# 7. UX Architecture

## 7.1 Layout Model

The tab should use a **browser workbench layout** with three layers:

### Header / Toolbar Layer

Persistent browser-like control area.

### Main Viewport Layer

Primary rendering area for the embedded preview.

### Context / Metadata Layer

Secondary panel or footer section for source/build status and diagnostics.

---

# 8. UI Spec

## 8.1 Toolbar

The Preview toolbar should feel like a lightweight browser control strip.

### Required elements

- page title: `Preview`
- preview status badge
- source run badge
- preview URL or route label
- device mode switcher
- refresh state button
- reload frame button
- open in new tab button
- copy preview URL button

### Toolbar goals

- make source and state explicit
- reduce navigation friction
- reinforce that the tab is tied to a live build output

### Suggested controls

Left side:

- `Preview`
- status pill
- run id pill

Center:

- URL/route display field

Right side:

- Desktop / Tablet / Mobile toggle
- Refresh
- Reload
- Open External
- Copy Link

---

## 8.2 Main Preview Viewport

This is the main visual container for the built output.

### Required behavior

- render an iframe or embedded webview when preview is ready
- support internal loading spinner
- support load failure overlay
- support viewport width constraints
- preserve chosen viewport mode while status refreshes

### Device modes

#### Desktop

- full width
- minimal frame chrome

#### Tablet

- centered constrained width
- medium-width frame

#### Mobile

- narrow centered portrait frame
- optional device-shell visual treatment

---

## 8.3 Metadata Panel

A side panel on wide screens, or a stacked footer panel on smaller screens.

### Required fields

- run id
- assembly id
- preview status
- build status
- preview URL
- entry route
- generated at
- last refreshed at
- environment/target
- embeddable yes/no
- failure reason if present

### Panel purpose

This panel makes the preview auditable and operationally trustworthy.

---

## 8.4 Empty / Transitional States

The tab must be useful even when it cannot yet show a preview.

### State cards required

- no run yet
- no preview generated
- build running
- preview preparing
- build failed
- preview load failed
- preview expired/unavailable

Each card must include:

- a short explanation
- a next action
- relevant status context

---

# 9. Behavioral State Spec

## 9.1 No Run Exists

### Trigger

Assembly has no buildable run / no preview-capable run.

### UI

- empty-state card
- supportive explanation
- CTA to move to Build or run pipeline

### Actions

- `Go to Build`
- `Run Pipeline` if allowed by page context

The page already has run controls and run-aware assembly state, so this state can point directly into that existing flow.

---

## 9.2 Build Running

### Trigger

Assembly status is running, or preview status reports active preparation.

### UI

- prominent loading state
- build/run id visible
- progress text if available
- polling indicator

### Actions

- `Refresh`
- `Go to Build`

This aligns with the current page behavior, which already polls assembly data while status is `running`.

assembly

---

## 9.3 Preview Preparing

### Trigger

Build is complete, but preview URL is not yet active.

### UI

- “Preview is preparing” card
- timestamp of last status check
- note that build output exists but preview endpoint is not ready

### Actions

- `Refresh`
- `Reload`
- `Open Build`

---

## 9.4 Preview Ready

### Trigger

Preview status resolves to ready and returns a valid preview URL.

### UI

- iframe/webview visible
- toolbar active
- metadata panel populated

### Actions

- `Reload Preview`
- `Open in New Tab`
- `Copy Link`
- device toggle

---

## 9.5 Build Failed

### Trigger

Build process failed before a preview could be published.

### UI

- failure banner/card
- concise error text
- source run id
- timestamp

### Actions

- `Open Build`
- `Inspect Logs`
- `Retry / Re-run`

---

## 9.6 Preview Load Failure

### Trigger

A preview URL exists, but the embedded frame fails to load.

### Common causes

- iframe/CSP restrictions
- expired preview session
- preview service unresponsive
- network issues
- invalid preview URL

### UI

- load-failure overlay
- explanation distinct from build failure
- show external-open option prominently

### Actions

- `Reload`
- `Open in New Tab`

- `Refresh Preview State`

---

# 10. Data Contract Spec

## 10.1 Frontend Contract

The Preview tab should not infer preview readiness from artifact files. It should use a dedicated preview status contract.

### Suggested response

```
type AssemblyPreviewStatus = {
 status: "none" | "building" | "preparing" | "ready" | "failed" | "expired";
 runId: string | null;
 buildStatus: string | null;
 previewUrl: string | null;
 entryUrl: string | null;
 updatedAt: string | null;
 generatedAt?: string | null;
 expiresAt?: string | null;
 embeddable?: boolean;
 environment?: string | null;
 error?: string | null;
};
```

### Field meanings

- `status`: current preview lifecycle state
- `runId`: source run
- `buildStatus`: build pipeline result
- `previewUrl`: embeddable or externally openable URL
- `entryUrl`: root route or default path
- `updatedAt`: last preview-state update
- `generatedAt`: when preview artifact/session was generated
- `expiresAt`: session expiry if ephemeral
- `embeddable`: whether iframe load is expected to work
- `environment`: runtime target label
- `error`: short operational reason

---

## 10.2 API Endpoints

### Required

`GET /api/assemblies/:id/preview`

Returns the current preview state for the assembly.

### Optional but recommended

`POST /api/assemblies/:id/preview/refresh`

Forces preview readiness re-resolution.

### Optional later

`POST /api/assemblies/:id/preview/session`
Creates or renews ephemeral preview sessions.

---

# 11. Resolution Rules

## 11.1 Which Preview to Show

Default to the latest valid preview-capable run.

### Recommended resolution order

1. latest run with `preview ready`
2. latest successful buildable run with preview preparing
3. current active run if it is generating preview
4. otherwise return `none`

### Rule

Never show stale preview content without also showing the source run and update time.

---

## 11.2 Embedding Rules

### If `embeddable === true`

Load preview in iframe.

### If `embeddable === false`

Do not attempt embedded rendering as the primary experience.
Show:

- non-embeddable message
- external launch button
- source metadata

---

# 12. Component Tree

## 12.1 `AssemblyPreviewTab`

Top-level controller component for the tab.

### Responsibilities

- fetch preview status
- poll while needed
- manage viewport mode
- manage iframe reload token
- render state-specific content
- pass controls to child components

### Props

```
type AssemblyPreviewTabProps = {
 assemblyId: number;
 runId: string | null;
 pipelineStatus: string;
};
```

---

## 12.2 `PreviewToolbar`

### Responsibilities

- display title, status, run badge, URL
- viewport toggle
- refresh/reload/open/copy actions

---

## 12.3 `PreviewViewport`

### Responsibilities

- embed iframe/webview
- apply mode-specific width shell
- show internal load spinner
- detect and surface frame load failure

### Internal state

- loading
- loaded
- loadError

---

## 12.4 `PreviewMetaPanel`

### Responsibilities

- display run/build source info
- display timestamps and environment
- display embeddable state
- display concise diagnostics

---

## 12.5 `PreviewStateCard`

Reusable empty/error/transitional state component.

### Variants

- none
- building

- preparing
- failed
- expired
- loadError
- nonEmbeddable

---

# 13. Query / State Management

## 13.1 Query Strategy

The assembly page already uses React Query and page-level `apiRequest`, so Preview should follow the same pattern instead of introducing a separate fetch model.

### Suggested query

```
useQuery({
 queryKey: ["/api/assemblies", assemblyId, "preview"],
 queryFn: () => apiRequest(`/api/assemblies/${assemblyId}/preview`),
 enabled: !!assemblyId,
 refetchInterval: (query) => {
   const d = query.state.data as AssemblyPreviewStatus | undefined;
   return d?.status === "building" || d?.status === "preparing" || pipelineStatus === "running"
     ? 2000
     : false;
 },
});
```

### Why

This matches the current polling pattern already used on the page for active pipeline work.

assembly

---

## 13.2 Local Component State

`AssemblyPreviewTab` should keep only UI-local state:

- `deviceMode: "desktop" | "tablet" | "mobile"`
- `reloadNonce: number`

- `copied: boolean`
- `frameState: "idle" | "loading" | "ready" | "error"`

---

# 14. Actions Spec

## 14.1 Refresh Preview State

### Purpose

Refetch preview status from backend.

### Effect

- re-evaluate preview availability
- update metadata
- update frame URL if changed

---

## 14.2 Reload Preview Frame

### Purpose

Reload the iframe content without forcing a full React Query refresh.

### Effect

- increment iframe key or reload token
- preserve mode and metadata state

---

## 14.3 Open in New Tab

### Purpose

Allow full-browser interaction.

### Behavior

- use `previewUrl`
- open in a separate tab/window
- available whenever preview URL exists

---

## 14.4 Copy Link

### Purpose

Fast sharing and quick manual testing.

### Behavior

- copies `previewUrl`
- toast confirmation on success

The page already uses `toast` via `sonner`, so Preview should reuse that pattern.

---

# 15. Integration Spec in `assembly.tsx`

## 15.1 Current Condition

The current `activeTab === "preview"` branch renders a placeholder `GlassPanel` with “Coming Soon” content.

assembly

## 15.2 Required Change

Replace that branch with the real component:

```
{activeTab === "preview" && (
 <AssemblyPreviewTab
   assemblyId={Number(id)}
   runId={assembly.runId}
   pipelineStatus={assembly.status}
 />
)}
```

## 15.3 Why This Fits

This aligns with how the page already mounts tab-specific components such as `IntakeEditor`, `BuildTabWithRuns`, and `ConfigTab`.

assembly

---

# 16. Visual Design Spec

## 16.1 Design Intent

Preview should look like a **browser lab** inside the assembly page.

### Style cues

- retain `GlassPanel` language already used across the page
- browser-like toolbar at top
- large clean viewing region
- subtle borders and state pills
- centered constrained shells for tablet/mobile modes

The page is already using `GlassPanel` and a polished operator-style visual system, so Preview should feel native to that ecosystem rather than like an embedded foreign utility.

---

## 16.2 Responsive Behavior

### Wide screens

- toolbar full width
- metadata side panel on the right

### Medium screens

- toolbar wraps
- metadata panel collapses below viewport

### Small screens

- stack controls
- compact URL field
- mobile preview still supported but with tighter chrome

---

# 17. Error Handling Spec

## 17.1 Build Failure vs Preview Failure

These must remain separate.

### Build failure

Means the build process did not produce a usable preview.

### Preview failure

Means a preview target exists, but the app cannot display it correctly.

### UI consequence

They should have different messages, icons, and recommended actions.

---

## 17.2 Invalid / Missing URL

If backend returns `ready` but no URL, coerce UI into a guarded error state:

- “Preview marked ready but no preview URL was returned.”

---

## 17.3 Expired Session

If preview URLs are ephemeral:

- detect expiry
- show expired state
- offer refresh/regenerate action

---

## 18. Accessibility / Interaction Rules

- all toolbar actions must be keyboard reachable
- device toggles should be button group with visible active state
- iframe fallback state must be readable without relying on color alone
- status indicators must include text labels
- URL field should be selectable/copyable

---

## 19. Acceptance Criteria

### Functional

- opening Preview shows a real preview when one exists
- preview always shows source run context
- user can switch desktop/tablet/mobile modes
- user can refresh preview state
- user can reload embedded preview
- user can open preview in a new tab
- no-preview/building/failed states are all supported

### UX

- Preview does not resemble Artifacts
- source/run freshness is always visible
- failure states are actionable, not generic
- iframe load failure is distinct from build failure

### Technical

- integrates into existing `activeTab === "preview"` branch
- uses existing query patterns and `apiRequest`
- supports polling while preview/build is active
- does not break the rest of the assembly page tab model

---

## 20. Implementation Checklist

## Phase 1 — Component Skeleton

- create `AssemblyPreviewTab`
- create `PreviewToolbar`
- create `PreviewViewport`
- create `PreviewMetaPanel`
- create `PreviewStateCard`

## Phase 2 — Data Wiring

- add `GET /api/assemblies/:id/preview`
- wire React Query fetch
- add polling logic
- map response fields to UI

## Phase 3 — Runtime Behavior

- embed iframe
- add mode toggles
- add refresh and reload
- add open external and copy link

## Phase 4 — States

- no preview
- building
- preparing
- ready
- failed
- expired
- loadError
- nonEmbeddable

## Phase 5 — Hardening

- distinguish frame failure from build failure
- preserve device mode during refetch
- support stale/changed preview URLs
- refine mobile/tablet sizing

---

# 21. Recommended Next Deliverable

The strongest next step is the **component spec pack**, which would include:

- exact component tree
- prop interfaces
- state model
- query model
- JSX section layout
- implementation checklist per component

# Assembly Preview Tab — Component Spec Pack

# Assembly Preview Tab — Component Spec Pack

## 1. Integration Target

### Existing insertion point

Replace this current branch:

```
{activeTab === "preview" && (
 <GlassPanel solid>
   ...
 </GlassPanel>
)}
```

with a real mounted component:

```
{activeTab === "preview" && (
 <AssemblyPreviewTab
   assemblyId={Number(id)}
   runId={assembly.runId}
   pipelineStatus={assembly.status}
 />
)}
```

That matches how the page already mounts dedicated tab components like `IntakeEditor`, `ArtifactBrowser`, and `BuildTabWithRuns`.

---

## 2. Component Tree

## 2.1 Top-Level

### `AssemblyPreviewTab`

Root controller for the preview experience.

**Responsibilities**

- fetch preview status
- manage polling while build/preview is active
- manage viewport mode
- manage reload token for iframe refresh
- render correct state branch
- coordinate toolbar, viewport, and metadata panel

**Props**

```
type AssemblyPreviewTabProps = {
 assemblyId: number;
 runId: string | null;
 pipelineStatus: string;
};
```

---

## 2.2 Child Components

### `PreviewToolbar`

Browser-like header strip.

**Responsibilities**

- show title
- show preview status badge
- show run badge
- show URL display
- allow viewport switching
- allow refresh/reload/open/copy actions

**Props**

```
type PreviewToolbarProps = {
 status: PreviewStatusKind;
 runId: string | null;
 previewUrl: string | null;
 deviceMode: PreviewDeviceMode;
 onSetDeviceMode: (mode: PreviewDeviceMode) => void;
 onRefreshState: () => void;
 onReloadFrame: () => void;
 onOpenExternal: () => void;
 onCopyLink: () => void;
 busy?: boolean;
};
```

---

## PreviewViewport

Main embedded preview area.

**Responsibilities**

- render iframe when preview is ready
- render device-shell layout
- show loading overlay
- handle iframe load errors
- preserve current width mode

**Props**

```
type PreviewViewportProps = {
 status: PreviewStatusKind;
 previewUrl: string | null;
 deviceMode: PreviewDeviceMode;
 reloadNonce: number;
 embeddable?: boolean;
 onFrameLoad?: () => void;
 onFrameError?: () => void;
};
```

---

## PreviewMetaPanel

Operational context panel.

**Responsibilities**

- show source/build metadata
- expose preview audit details
- separate build state from frame state
- show timestamps and environment

**Props**

```
type PreviewMetaPanelProps = {
 assemblyId: number;
 data: AssemblyPreviewStatus | null;
 frameState: PreviewFrameState;
};
```

---

## PreviewStateCard

Reusable card for empty, loading, failed, and unsupported cases.

**Responsibilities**

- render concise state-specific message
- provide actions for next step
- keep placeholder states visually consistent

**Props**

```
type PreviewStateCardProps = {
 variant:
   | "none"
   | "building"
   | "preparing"
   | "failed"
   | "expired"
   | "loadError"
   | "nonEmbeddable";
 title: string;
 description: string;
 actions?: React.ReactNode;
};
```

---

# 3. Type Model

## 3.1 Status Types

```
type PreviewStatusKind =
 | "none"
 | "building"
 | "preparing"
 | "ready"
 | "failed"
 | "expired";
```

## 3.2 Device Modes

```
type PreviewDeviceMode = "desktop" | "tablet" | "mobile";
```

## 3.3 Frame State

```
type PreviewFrameState = "idle" | "loading" | "ready" | "error";
```

## 3.4 API Response

```
type AssemblyPreviewStatus = {
 status: PreviewStatusKind;
 runId: string | null;
 buildStatus: string | null;
 previewUrl: string | null;
 entryUrl: string | null;
 updatedAt: string | null;
 generatedAt?: string | null;
 expiresAt?: string | null;
 embeddable?: boolean;
 environment?: string | null;
 error?: string | null;
};
```

---

# 4. Query Model

## 4.1 Query Key

Use the same style the page already uses with assembly-scoped keys. The current page query pattern is `["/api/assemblies", id]`, so Preview should stay aligned with that convention.

assembly

queryKey: ["/api/assemblies", assemblyId, "preview"]

## 4.2 Query Function

queryFn: () => apiRequest(`/api/assemblies/${assemblyId}/preview`)

## 4.3 Polling Logic

Preview should poll when:

- assembly pipeline is running
- preview status is `building`
- preview status is `preparing`

```
refetchInterval: (query) => {
 const d = query.state.data as AssemblyPreviewStatus | undefined;
 return d?.status === "building" ||
```

```
  d?.status === "preparing" ||
  pipelineStatus === "running"
  ? 2000
  : false;
}
```

This mirrors the existing page behavior where the assembly query already refetches every 2 seconds when status is `running`.

assembly

---

# 5. Local State Model

Inside `AssemblyPreviewTab`, keep only UI-local state.

```
const [deviceMode, setDeviceMode] = useState<PreviewDeviceMode>("desktop");
const [reloadNonce, setReloadNonce] = useState(0);
const [frameState, setFrameState] = useState<PreviewFrameState>("idle");
```

### State meanings

- `deviceMode`: current viewport preset
- `reloadNonce`: forces iframe remount when reloading preview
- `frameState`: tracks embedded frame lifecycle independent of backend preview status

---

# 6. Action Model

# 6.1 Refresh Preview State

Refetch preview status from backend.

```
const handleRefreshState = async () => {
 await refetch();
};
```

### Purpose

- re-resolve preview URL
- update preview readiness

- update metadata

---

## 6.2 Reload Frame

Reload only the iframe.

```
const handleReloadFrame = () => {
 setFrameState("loading");
 setReloadNonce((n) => n + 1);
};
```

### Purpose

- preserve fetched preview data
- avoid unnecessary outer query churn

---

## 6.3 Open External

```
const handleOpenExternal = () => {
 if (!data?.previewUrl) return;
 window.open(data.previewUrl, "_blank", "noopener,noreferrer");
};
```

---

## 6.4 Copy Link

```
const handleCopyLink = async () => {
 if (!data?.previewUrl) return;
 await navigator.clipboard.writeText(data.previewUrl);
 toast.success("Preview link copied");
};
```

The page already uses `toast`, so Preview should reuse that same interaction pattern.

assembly

---

# 7. Rendering Rules

## 7.1 Top-Level Branching

`AssemblyPreviewTab` should branch in this order:

1. query loading
2. query error
3. `status === "none"`
4. `status === "building"`
5. `status === "preparing"`
6. `status === "failed"`
7. `status === "expired"`
8. `status === "ready"` with `embeddable === false`
9. `status === "ready"` normal viewport
10. fallback invalid state

---

## 7.2 Frame-State Branching

Within `ready` state:

- show iframe shell immediately
- show overlay while frameState is `loading`
- show load-error overlay when frameState is `error`

This is important because a backend-ready preview can still fail in iframe load.

---

# 8. Layout Spec

## 8.1 Outer Shell

Use `GlassPanel solid` as the main shell so it visually matches the rest of the page, which already leans heavily on `GlassPanel` across tabs and side panels.

### Structure

```
<GlassPanel solid className="overflow-hidden">
 <div className="border-b ...">
   <PreviewToolbar ... />
```

```
 </div>

 <div className="grid gap-4 lg:grid-cols-[minmax(0,1fr)_320px] p-4">
   <PreviewViewport ... />
   <PreviewMetaPanel ... />
 </div>
</GlassPanel>
```

---

## 8.2 Toolbar Layout

### Left cluster

- Monitor icon
- `Preview`
- status pill
- run pill

### Center

- URL display / route field

### Right cluster

- Desktop / Tablet / Mobile segmented buttons
- Refresh
- Reload
- Open
- Copy

---

## 8.3 Viewport Layout

The viewport should fill most of the available space and use a centered device wrapper.

### Desktop wrapper

max-w-none w-full

### Tablet wrapper

max-w-4xl mx-auto

### Mobile wrapper

max-w-[420px] mx-auto

### Minimum viewport height

Use something like:

min-h-[70vh]

---

## 8.4 Metadata Layout

Use a vertical card stack:

- preview source
- runtime status
- timestamps
- environment
- diagnostics

---

# 9. JSX Skeleton

## 9.1 `AssemblyPreviewTab`

```
function AssemblyPreviewTab({ assemblyId, runId, pipelineStatus }: AssemblyPreviewTabProps)
{
 const [deviceMode, setDeviceMode] = useState<PreviewDeviceMode>("desktop");
 const [reloadNonce, setReloadNonce] = useState(0);
 const [frameState, setFrameState] = useState<PreviewFrameState>("idle");

 const { data, isLoading, error, refetch, isFetching } = useQuery<AssemblyPreviewStatus>({
   queryKey: ["/api/assemblies", assemblyId, "preview"],
   queryFn: () => apiRequest(`/api/assemblies/${assemblyId}/preview`),
   enabled: !!assemblyId,
   refetchInterval: (query) => {
     const d = query.state.data as AssemblyPreviewStatus | undefined;
     return d?.status === "building" || d?.status === "preparing" || pipelineStatus === "running"
       ? 2000
       : false;
   },
 });
```

```
useEffect(() => {
  if (data?.status === "ready") {
    setFrameState("loading");
  } else {
    setFrameState("idle");
  }
}, [data?.previewUrl, data?.status, reloadNonce]);

const handleRefreshState = async () => {
  await refetch();
};

const handleReloadFrame = () => {
  setFrameState("loading");
  setReloadNonce((n) => n + 1);
};

const handleOpenExternal = () => {
  if (!data?.previewUrl) return;
  window.open(data.previewUrl, "_blank", "noopener,noreferrer");
};

const handleCopyLink = async () => {
  if (!data?.previewUrl) return;
  await navigator.clipboard.writeText(data.previewUrl);
  toast.success("Preview link copied");
};

return (
  <GlassPanel solid className="overflow-hidden">
    <PreviewToolbar
      status={data?.status ?? "none"}
      runId={data?.runId ?? runId}
      previewUrl={data?.previewUrl ?? null}
      deviceMode={deviceMode}
      onSetDeviceMode={setDeviceMode}
      onRefreshState={handleRefreshState}
      onReloadFrame={handleReloadFrame}
      onOpenExternal={handleOpenExternal}
      onCopyLink={handleCopyLink}
      busy={isFetching}
    />
```

```
    <div className="grid gap-4 p-4 lg:grid-cols-[minmax(0,1fr)_320px]">
      <div className="min-w-0">
        {/* state branch goes here */}
      </div>

      <PreviewMetaPanel
        assemblyId={assemblyId}
        data={data ?? null}
        frameState={frameState}
      />
    </div>
  </GlassPanel>
);
}
```

---

## 9.2 State Branch Body

```
if (isLoading) {
 return <PreviewStateCard variant="building" title="Loading preview state"
description="Checking for an available preview target." />;
}

if (error) {
 return <PreviewStateCard variant="failed" title="Preview state failed to load" description="The
preview service could not be queried." />;
}

if (!data || data.status === "none") {
 return <PreviewStateCard variant="none" title="No preview available" description="Run the
pipeline or build to generate a preview." />;
}

if (data.status === "building") {
 return <PreviewStateCard variant="building" title="Build in progress" description="The preview
will appear when the build is ready." />;
}

if (data.status === "preparing") {
 return <PreviewStateCard variant="preparing" title="Preview is preparing" description="The
build completed, and the preview target is being initialized." />;
}

if (data.status === "failed") {
```

```
 return <PreviewStateCard variant="failed" title="Build failed" description={data.error || "No preview could be generated from the current run."} />;
}

if (data.status === "expired") {
 return <PreviewStateCard variant="expired" title="Preview expired" description="This preview session is no longer active. Refresh to resolve a new session." />;
}

if (data.status === "ready" && data.embeddable === false) {
 return <PreviewStateCard variant="nonEmbeddable" title="Preview cannot be embedded" description="Open it in a new tab to inspect the build." />;
}

return (
 <PreviewViewport
   status={data.status}
   previewUrl={data.previewUrl}
   deviceMode={deviceMode}
   reloadNonce={reloadNonce}
   embeddable={data.embeddable}
   onFrameLoad={() => setFrameState("ready")}
   onFrameError={() => setFrameState("error")}
 />
);
```

---

# 10. `PreviewToolbar` Spec

## 10.1 Visual Structure

Single-row on large screens, wrapped on smaller screens.

### Left

- icon
- title
- status badge
- run badge

### Middle

- read-only URL display

**Right**

- segmented mode control
- icon buttons / text buttons

## 10.2 Needed Controls

- `Desktop`
- `Tablet`
- `Mobile`
- `Refresh`
- `Reload`
- `Open`
- `Copy`

## 10.3 Status Badge Mapping

```
const statusTone = {
 none: "muted",
 building: "cyan",
 preparing: "amber",
 ready: "green",
 failed: "red",
 expired: "violet",
};
```

---

# 11. `PreviewViewport` Spec

## 11.1 Wrapper Logic

```
const modeClass =
 deviceMode === "desktop"
   ? "w-full"
   : deviceMode === "tablet"
   ? "max-w-4xl mx-auto"
   : "max-w-[420px] mx-auto";
```

## 11.2 Iframe Element

```
<iframe
```

```
 key={`${previewUrl}-${reloadNonce}`}
 src={previewUrl ?? undefined}
 title="Assembly Preview"
 className="w-full min-h-[70vh] rounded-xl border border-[hsl(var(--border))] bg-white"
 onLoad={onFrameLoad}
 onError={onFrameError}
/>
```

## 11.3 Overlay States

Show overlays for:

- frame loading
- frame error

### Frame loading overlay

Centered spinner + “Loading preview…”

### Frame error overlay

Message:

- “The preview could not be displayed in the embedded viewer.”

Primary action should be:

- Open in new tab

---

# 12. `PreviewMetaPanel` Spec

## 12.1 Sections

### Section A — Source

- Assembly ID
- Run ID
- Status
- Build Status

### Section B — Preview Target

- Preview URL
- Entry URL
- Embeddable
- Environment

### Section C — Timing

- Generated At
- Updated At
- Expires At

### Section D — Diagnostics

- Frame State
- Error
- Ready vs embedded state mismatch

## 12.2 Row Pattern

Use the same compact label/value presentation style already used elsewhere in the page config/info panels.

assembly

---

# 13. `PreviewStateCard` Spec

## 13.1 Shared Layout

- icon
- title
- description
- optional action row

## 13.2 Icon Mapping

- none → `Monitor`
- building → `Loader2`
- preparing → `Clock3`
- failed → `AlertTriangle`

- expired → `TimerReset`
- loadError → `WifiOff`
- nonEmbeddable → `ExternalLink`

---

# 14. Styling Rules

## 14.1 Keep Native to Current Page

Use:

- `GlassPanel`
- existing muted/foreground/border tokens
- existing badge language
- existing rounded panel shapes

The current page already uses this visual system consistently, including tab panels and inspector cards.

## 14.2 Do Not Style It Like Artifacts

Avoid:

- file grids
- output tiles
- document cards
- code panes as primary surface

This directly corrects the current placeholder messaging, which is still framed around “generated artifacts and documents.”

assembly

---

# 15. File / Module Plan

### Minimal first pass

If you want to move fast, define these inside `assembly.tsx` near `BuildTabWithRuns`:

- `AssemblyPreviewTab`
- `PreviewToolbar`
- `PreviewViewport`
- `PreviewMetaPanel`
- `PreviewStateCard`

### Better long-term split

- `App/src/components/assembly/AssemblyPreviewTab.tsx`
- `App/src/components/assembly/PreviewToolbar.tsx`
- `App/src/components/assembly/PreviewViewport.tsx`
- `App/src/components/assembly/PreviewMetaPanel.tsx`
- `App/src/components/assembly/PreviewStateCard.tsx`

---

## 16. Implementation Order

### Pass 1

- replace placeholder branch
- create `AssemblyPreviewTab`
- wire preview query
- add `PreviewStateCard`

### Pass 2

- add `PreviewToolbar`
- add device toggles
- add refresh / reload / open / copy

### Pass 3

- add `PreviewViewport`
- add iframe load/error handling
- add frame overlays

### Pass 4

- add `PreviewMetaPanel`
- finish status rows and diagnostics

### Pass 5

- harden empty/error states
- refine mobile/tablet shells
- add embeddable fallback

---

## 17. Done Definition

This component pack is complete when:

- the placeholder Preview panel is gone
- the tab renders a real browser-style preview workspace
- the preview is tied to assembly/build state
- the user can inspect desktop/tablet/mobile
- the user can refresh, reload, open, and copy
- unavailable and failed states are explicit and useful

# Assembly Preview Tab -- Assembly Preview Tab

Next is the **actual implementation scaffold** for the Preview tab inside `assembly.tsx`, replacing the current placeholder branch. That branch is currently still a “Coming Soon” `GlassPanel`, and the page already has the right patterns in place: `useQuery`, `apiRequest`, `toast`, `GlassPanel`, `Monitor`, and run-aware polling on the assembly query.

assembly

assembly

assembly

## 1. Replace the current Preview tab branch

Replace the existing `activeTab === "preview"` block with this:

```
{activeTab === "preview" && (
 <AssemblyPreviewTab
   assemblyId={Number(id)}
   runId={assembly.runId}
   pipelineStatus={assembly.status}
 />
)}
```

That directly swaps out the current placeholder panel.

assembly

---

## 2. Add the types

Put these below your other local constants/helpers:

```
type PreviewStatusKind =
 | "none"
 | "building"
 | "preparing"
 | "ready"
 | "failed"
 | "expired";
```

```
type PreviewDeviceMode = "desktop" | "tablet" | "mobile";

type PreviewFrameState = "idle" | "loading" | "ready" | "error";

type AssemblyPreviewStatus = {
 status: PreviewStatusKind;
 runId: string | null;
 buildStatus: string | null;
 previewUrl: string | null;
 entryUrl: string | null;
 updatedAt: string | null;
 generatedAt?: string | null;
 expiresAt?: string | null;
 embeddable?: boolean;
 environment?: string | null;
 error?: string | null;
};
```

---

## 3. Add the main Preview tab component

Put this near `BuildTabWithRuns` so it stays in the same local-component pattern the file already uses.

assembly

```
function AssemblyPreviewTab({
 assemblyId,
 runId,
 pipelineStatus,
}: {
 assemblyId: number;
 runId: string | null;
 pipelineStatus: string;
}) {
 const [deviceMode, setDeviceMode] = useState<PreviewDeviceMode>("desktop");
 const [reloadNonce, setReloadNonce] = useState(0);
 const [frameState, setFrameState] = useState<PreviewFrameState>("idle");

 const {
   data,
   isLoading,
   error,
```

```
  isFetching,
  refetch,
} = useQuery<AssemblyPreviewStatus>({
  queryKey: ["/api/assemblies", assemblyId, "preview"],
  queryFn: () => apiRequest(`/api/assemblies/${assemblyId}/preview`),
  enabled: !!assemblyId,
  refetchInterval: (query) => {
    const d = query.state.data as AssemblyPreviewStatus | undefined;
    return d?.status === "building" ||
      d?.status === "preparing" ||
      pipelineStatus === "running"
      ? 2000
      : false;
  },
});

useEffect(() => {
  if (data?.status === "ready" && data?.previewUrl) {
    setFrameState("loading");
  } else {
    setFrameState("idle");
  }
}, [data?.status, data?.previewUrl, reloadNonce]);

const handleRefreshState = async () => {
  await refetch();
};

const handleReloadFrame = () => {
  setFrameState("loading");
  setReloadNonce((n) => n + 1);
};

const handleOpenExternal = () => {
  if (!data?.previewUrl) return;
  window.open(data.previewUrl, "_blank", "noopener,noreferrer");
};

const handleCopyLink = async () => {
  if (!data?.previewUrl) return;
  try {
    await navigator.clipboard.writeText(data.previewUrl);
    toast.success("Preview link copied");
  } catch {
```

```
    toast.error("Failed to copy preview link");
  }
};

return (
  <GlassPanel solid className="overflow-hidden">
    <PreviewToolbar
      status={data?.status ?? "none"}
      runId={data?.runId ?? runId}
      previewUrl={data?.previewUrl ?? null}
      deviceMode={deviceMode}
      onSetDeviceMode={setDeviceMode}
      onRefreshState={handleRefreshState}
      onReloadFrame={handleReloadFrame}
      onOpenExternal={handleOpenExternal}
      onCopyLink={handleCopyLink}
      busy={isFetching}
    />

    <div className="grid gap-4 p-4 lg:grid-cols-[minmax(0,1fr)_320px]">
      <div className="min-w-0">
        {isLoading ? (
          <PreviewStateCard
            icon={<Loader2 className="w-5 h-5 animate-spin" />}
            title="Loading preview state"
            description="Checking for an available build preview."
          />
        ) : error ? (
          <PreviewStateCard
            icon={<AlertTriangle className="w-5 h-5" />}
            title="Preview state failed to load"
            description="The preview service could not be queried."
            actions={
              <button
                onClick={() => refetch()}
                className="px-3 py-2 rounded-md text-xs font-medium border
border-[hsl(var(--border))] hover:bg-[hsl(var(--muted))]/40"
              >
                Retry
              </button>
            }
          />
        ) : !data || data.status === "none" ? (
          <PreviewStateCard
```

```
          icon={<Monitor className="w-5 h-5" />}
          title="No preview available"
          description="Run the pipeline or build flow to generate a live preview."
        />
      ) : data.status === "building" ? (
        <PreviewStateCard
          icon={<Loader2 className="w-5 h-5 animate-spin" />}
          title="Build in progress"
          description="The preview will appear when the build is ready."
        />
      ) : data.status === "preparing" ? (
        <PreviewStateCard
          icon={<Clock className="w-5 h-5" />}
          title="Preview is preparing"
          description="The build completed and the preview target is being initialized."
        />
      ) : data.status === "failed" ? (
        <PreviewStateCard
          icon={<AlertTriangle className="w-5 h-5" />}
          title="Build failed"
          description={data.error || "No preview could be generated from the current run."}
        />
      ) : data.status === "expired" ? (
        <PreviewStateCard
          icon={<Timer className="w-5 h-5" />}
          title="Preview expired"
          description="This preview session is no longer active. Refresh to resolve a new
session."
          actions={
            <button
              onClick={handleRefreshState}
              className="px-3 py-2 rounded-md text-xs font-medium border
border-[hsl(var(--border))] hover:bg-[hsl(var(--muted))]/40"
            >
              Refresh
            </button>
          }
        />
      ) : data.status === "ready" && data.embeddable === false ? (
        <PreviewStateCard
          icon={<ArrowUpCircle className="w-5 h-5" />}
          title="Preview cannot be embedded"
          description="Open the preview in a new tab to inspect the build."
          actions={
```

```
            <button
              onClick={handleOpenExternal}
              className="px-3 py-2 rounded-md text-xs font-medium border
border-[hsl(var(--border))] hover:bg-[hsl(var(--muted))]/40"
            >
              Open Preview
            </button>
          }
        />
      ) : (
        <PreviewViewport
          previewUrl={data.previewUrl}
          deviceMode={deviceMode}
          reloadNonce={reloadNonce}
          frameState={frameState}
          onFrameLoad={() => setFrameState("ready")}
          onFrameError={() => setFrameState("error")}
          onOpenExternal={handleOpenExternal}
        />
      )}
    </div>

    <PreviewMetaPanel
      assemblyId={assemblyId}
      data={data ?? null}
      frameState={frameState}
    />
  </div>
 </GlassPanel>
);
}
```

---

# 4. Add the toolbar

```
function PreviewToolbar({
 status,
 runId,
 previewUrl,
 deviceMode,
 onSetDeviceMode,
 onRefreshState,
 onReloadFrame,
 onOpenExternal,
```

```
 onCopyLink,
 busy = false,
}: {
 status: PreviewStatusKind;
 runId: string | null;
 previewUrl: string | null;
 deviceMode: PreviewDeviceMode;
 onSetDeviceMode: (mode: PreviewDeviceMode) => void;
 onRefreshState: () => void;
 onReloadFrame: () => void;
 onOpenExternal: () => void;
 onCopyLink: () => void;
 busy?: boolean;
}) {
 const statusClasses: Record<PreviewStatusKind, string> = {
   none: "bg-[hsl(var(--muted))]/40 text-[hsl(var(--muted-foreground))] border-[hsl(var(--border))]",
   building: "bg-cyan-500/10 text-cyan-300 border-cyan-500/30",
   preparing: "bg-amber-500/10 text-amber-300 border-amber-500/30",
   ready: "bg-emerald-500/10 text-emerald-300 border-emerald-500/30",
   failed: "bg-red-500/10 text-red-300 border-red-500/30",
   expired: "bg-violet-500/10 text-violet-300 border-violet-500/30",
 };

 return (
   <div className="border-b border-[hsl(var(--border))] p-4 space-y-3">
     <div className="flex flex-wrap items-center gap-3">
       <div className="flex items-center gap-2 min-w-0">
         <Monitor className="w-5 h-5 text-cyan-400" />
         <h2 className="text-sm font-semibold text-[hsl(var(--foreground))]">Preview</h2>
         <span className={`px-2 py-0.5 rounded-full border text-[10px] font-medium uppercase
tracking-wider ${statusClasses[status]}`}>
           {status}
         </span>
         <span className="px-2 py-0.5 rounded-full border border-[hsl(var(--border))] text-[10px]
font-medium uppercase tracking-wider text-[hsl(var(--muted-foreground))]">
           Run {runId || "—"}
         </span>
       </div>

       <div className="ml-auto flex flex-wrap items-center gap-2">
         {(["desktop", "tablet", "mobile"] as PreviewDeviceMode[]).map((mode) => (
           <button
             key={mode}
             onClick={() => onSetDeviceMode(mode)}
```

```
            className={`px-3 py-1.5 rounded-md text-xs border transition ${
              deviceMode === mode
                ? "bg-[hsl(var(--foreground))] text-[hsl(var(--background))]
border-[hsl(var(--foreground))]"
                : "border-[hsl(var(--border))] text-[hsl(var(--muted-foreground))]
hover:bg-[hsl(var(--muted))]/40"
            }`}
          >
            {mode[0].toUpperCase() + mode.slice(1)}
          </button>
        ))}

        <button
          onClick={onRefreshState}
          className="px-3 py-1.5 rounded-md text-xs border border-[hsl(var(--border))]
hover:bg-[hsl(var(--muted))]/40"
        >
          {busy ? "Refreshing..." : "Refresh"}
        </button>

        <button
          onClick={onReloadFrame}
          className="px-3 py-1.5 rounded-md text-xs border border-[hsl(var(--border))]
hover:bg-[hsl(var(--muted))]/40"
        >
          Reload
        </button>

        <button
          onClick={onOpenExternal}
          disabled={!previewUrl}
          className="px-3 py-1.5 rounded-md text-xs border border-[hsl(var(--border))]
hover:bg-[hsl(var(--muted))]/40 disabled:opacity-50"
        >
          Open
        </button>

        <button
          onClick={onCopyLink}
          disabled={!previewUrl}
          className="px-3 py-1.5 rounded-md text-xs border border-[hsl(var(--border))]
hover:bg-[hsl(var(--muted))]/40 disabled:opacity-50"
        >
          Copy Link
```

```
        </button>
      </div>
    </div>

    <div className="rounded-md border border-[hsl(var(--border))] bg-[hsl(var(--muted))]/20
px-3 py-2">
      <div className="text-[10px] uppercase tracking-wider text-[hsl(var(--muted-foreground))]
mb-1">
        Preview URL
      </div>
      <div className="text-xs font-mono-tech text-[hsl(var(--foreground))] break-all">
        {previewUrl || "No preview URL available"}
      </div>
    </div>
  </div>
 );
}
```

---

## 5. Add the viewport

```
function PreviewViewport({
 previewUrl,
 deviceMode,
 reloadNonce,
 frameState,
 onFrameLoad,
 onFrameError,
 onOpenExternal,
}: {
 previewUrl: string | null;
 deviceMode: PreviewDeviceMode;
 reloadNonce: number;
 frameState: PreviewFrameState;
 onFrameLoad: () => void;
 onFrameError: () => void;
 onOpenExternal: () => void;
}) {
 const modeClass =
   deviceMode === "desktop"
     ? "w-full"
     : deviceMode === "tablet"
     ? "max-w-4xl mx-auto"
     : "max-w-[420px] mx-auto";
```

```
 if (!previewUrl) {
   return (
     <PreviewStateCard
       icon={<AlertTriangle className="w-5 h-5" />}
       title="Preview URL missing"
       description="The preview is marked ready, but no preview URL was returned."
     />
   );
 }

 return (
   <div className="relative rounded-xl border border-[hsl(var(--border))]
bg-[hsl(var(--muted))]/20 p-4 min-h-[70vh]">
     <div className={modeClass}>
       <iframe
         key={`${previewUrl}-${reloadNonce}`}
         src={previewUrl}
         title="Assembly Preview"
         className="w-full min-h-[70vh] rounded-xl border border-[hsl(var(--border))] bg-white"
         onLoad={onFrameLoad}
         onError={onFrameError}
       />
     </div>

     {frameState === "loading" && (
       <div className="absolute inset-0 flex items-center justify-center rounded-xl bg-black/20
backdrop-blur-[1px]">
         <div className="flex items-center gap-2 px-4 py-2 rounded-md border
border-[hsl(var(--border))] bg-[hsl(var(--background))]">
           <Loader2 className="w-4 h-4 animate-spin" />
           <span className="text-xs text-[hsl(var(--foreground))]">Loading preview…</span>
         </div>
       </div>
     )}

     {frameState === "error" && (
       <div className="absolute inset-0 flex items-center justify-center rounded-xl bg-black/35
p-6">
         <div className="max-w-sm w-full rounded-xl border border-[hsl(var(--border))]
bg-[hsl(var(--background))] p-5 text-center space-y-3">
           <AlertTriangle className="w-6 h-6 mx-auto text-[hsl(var(--status-failure))]" />
           <div>
             <div className="text-sm font-semibold text-[hsl(var(--foreground))]">
```

```
            Embedded preview failed
          </div>
          <div className="text-xs text-[hsl(var(--muted-foreground))] mt-1">
            The preview could not be displayed in the embedded viewer.
          </div>
        </div>
        <button
          onClick={onOpenExternal}
          className="px-3 py-2 rounded-md text-xs font-medium border
border-[hsl(var(--border))] hover:bg-[hsl(var(--muted))]/40"
        >
          Open in New Tab
        </button>
      </div>
    </div>
  )}
 </div>
);
}
```

---

# 6. Add the metadata panel

This matches the compact label/value pattern already used elsewhere in the page.

assembly

```
function PreviewMetaPanel({
 assemblyId,
 data,
 frameState,
}: {
 assemblyId: number;
 data: AssemblyPreviewStatus | null;
 frameState: PreviewFrameState;
}) {
 const rows = [
   { label: "Assembly ID", value: String(assemblyId), mono: true },
   { label: "Run ID", value: data?.runId || "—", mono: true },
   { label: "Preview Status", value: data?.status || "—" },
   { label: "Build Status", value: data?.buildStatus || "—" },
   { label: "Frame State", value: frameState },
   { label: "Environment", value: data?.environment || "—" },
```

```
  { label: "Embeddable", value: data?.embeddable == null ? "—" : data.embeddable ? "Yes" :
"No" },
  { label: "Entry URL", value: data?.entryUrl || "—", mono: true },
  { label: "Updated At", value: data?.updatedAt || "—", mono: true },
  { label: "Generated At", value: data?.generatedAt || "—", mono: true },
  { label: "Expires At", value: data?.expiresAt || "—", mono: true },
 ];

 return (
  <div className="space-y-4">
   <GlassPanel solid className="p-4">
    <h3 className="text-system-label mb-3">Preview Context</h3>
    <div className="space-y-2">
     {rows.map((row) => (
      <div key={row.label} className="flex justify-between gap-4">
       <span className="text-xs text-[hsl(var(--muted-foreground))]">{row.label}</span>
       <span className={`text-xs text-right text-[hsl(var(--foreground))] ${row.mono ?
"font-mono-tech break-all" : ""}`}>
        {row.value}
       </span>
      </div>
     ))}
    </div>
   </GlassPanel>

   <GlassPanel solid className="p-4">
    <h3 className="text-system-label mb-3">Diagnostics</h3>
    <div className="space-y-2">
     <div className="text-xs text-[hsl(var(--muted-foreground))]">
      {data?.error || "No active diagnostic message."}
     </div>
    </div>
   </GlassPanel>
  </div>
 );
}
```

---

# 7. Add the shared state card

```
function PreviewStateCard({
 icon,
 title,
 description,
```

```
 actions,
}: {
 icon: React.ReactNode;
 title: string;
 description: string;
 actions?: React.ReactNode;
}) {
 return (
   <div className="min-h-[70vh] rounded-xl border border-[hsl(var(--border))]
bg-[hsl(var(--muted))]/20 flex items-center justify-center p-6">
     <div className="max-w-md w-full text-center space-y-4">
       <div className="mx-auto w-12 h-12 rounded-full border border-[hsl(var(--border))]
bg-[hsl(var(--background))] flex items-center justify-center text-[hsl(var(--foreground))]">
         {icon}
       </div>
       <div>
         <h3 className="text-base font-semibold text-[hsl(var(--foreground))]">{title}</h3>
         <p className="text-sm text-[hsl(var(--muted-foreground))] mt-2">{description}</p>
       </div>
       {actions ? <div className="flex justify-center">{actions}</div> : null}
     </div>
   </div>
 );
}
```

---

# 8. What this gives you immediately

After this pass, Preview becomes:

- a real runtime tab
- tied to assembly/build state
- viewport-aware
- able to refresh, reload, open, and copy
- separate from Artifacts and Build in purpose

That is the right correction because the current placeholder is still framed around “generated artifacts and documents,” not a true browser-style build preview.

assembly

# Assembly Preview — Backend Spec

# Assembly Preview — Backend Spec

## 1. Endpoint

### Primary read endpoint

`GET /api/assemblies/:id/preview`

Purpose:

- resolve the current preview state for an assembly
- identify the source run
- return the embeddable preview target if available
- distinguish build failure from preview failure
- support polling from the Preview tab

---

## 2. Response Contract

```
type PreviewStatusKind =
 | "none"
 | "building"
 | "preparing"
 | "ready"
 | "failed"
 | "expired";

type AssemblyPreviewResponse = {
 status: PreviewStatusKind;
 assemblyId: number;
 runId: string | null;
 buildStatus: string | null;
 previewUrl: string | null;
 entryUrl: string | null;
 updatedAt: string | null;
 generatedAt?: string | null;
 expiresAt?: string | null;
 embeddable?: boolean;
 environment?: string | null;
 source?: "active_run" | "latest_buildable_run" | "latest_ready_preview" | "none";
```

```
 error?: string | null;
};
```

---

## 3. Field Definitions

### `status`

The current lifecycle state of the preview.

Allowed values:

- `none` — no preview-capable run exists
- `building` — preview source build is still running
- `preparing` — build completed but preview target/session is still being created
- `ready` — preview target is available
- `failed` — build or preview generation failed
- `expired` — preview was ready before but session/target is no longer valid

### `assemblyId`

Echo of the requested assembly id.

### `runId`

The run currently used as the preview source.

### `buildStatus`

The build-specific status for the chosen run.

### `previewUrl`

The actual URL the frontend should embed or open externally.

### `entryUrl`

Optional root entry route if different from `previewUrl`.

### `updatedAt`

Last known status update time for the preview state.

### `generatedAt`

When the preview target/session was created.

### `expiresAt`

When the preview session stops being valid, if ephemeral.

### `embeddable`

Whether the frontend should expect iframe embedding to work.

### `environment`

Useful labels:

- `local`
- `staging`
- `preview`
- `sandbox`
- `production-like`

### `source`

Tells the frontend how the resolver chose this preview.

### `error`

Short operational reason for failure or non-ready state.

---

## 4. Resolver Inputs

The resolver should pull from these assembly-adjacent sources:

### 4.1 Assembly record

Needed for:

- assembly existence
- current `runId`

- current overall status

The assembly page already depends on assembly-level status and current run awareness, so Preview should use the same run-centric model rather than inventing its own detached state.

assembly

### 4.2 Buildable runs

Needed for:

- latest run with build metadata
- build completion/failure status
- whether a run has build output

The page already has a buildable-runs path in `BuildTabWithRuns`, which is the natural starting point for preview resolution.

assembly

### 4.3 Preview publication/session source

Needed for:

- preview URL
- embeddable flag
- expiry
- environment
- publication status

This may come from:

- build manifest
- deployment record
- preview session table
- runtime service registry
- local preview server mapping

---

## 5. Resolution Order

Use this order every time the endpoint is called:

### Step 1 — Validate assembly

If assembly does not exist:

- return 404

### Step 2 — Load preview-capable run candidates

Candidate priority:

1. assembly current run
2. latest buildable run
3. latest run with ready preview session

### Step 3 — Evaluate best candidate

Use the first candidate that can supply a meaningful preview state.

### Step 4 — Resolve preview lifecycle

Determine whether that candidate is:

- still building
- build-complete but preview-pending
- ready
- failed
- expired

### Step 5 — Return normalized response

Frontend should never need to guess.

---

## 6. Candidate Selection Rules

### 6.1 Prefer active current run when still running

If assembly has a current `runId` and assembly status is `running`:

- use that run as primary source
- if build stage not complete: return `building`
- if build complete but no preview URL yet: return `preparing`

This matches the existing page behavior, which already polls while assembly status is `running`.

assembly

### 6.2 Otherwise prefer latest successful buildable run

If the active run is not usable:

- take most recent run where build output exists or build status is known

`BuildTabWithRuns` already operates on “buildable” runs with fields like `runId`, `status`, `hasBuild`, and `buildStatus`, so the preview resolver should align with that same shape.

assembly

### 6.3 Otherwise use latest ready preview

If a prior run has a still-valid preview session:

- return that as `source = "latest_ready_preview"`

### 6.4 Otherwise return `none`

No candidate, no guessing.

---

## 7. Status Derivation Rules

## 7.1 `none`

Return `none` when:

- no runs exist
- runs exist but none are buildable
- no build output exists
- no preview source can be derived

Example:

```
{
 status: "none",
 assemblyId,
 runId: null,
 buildStatus: null,
```

```
 previewUrl: null,
 entryUrl: null,
 updatedAt: nowIso,
 source: "none",
 error: "No preview-capable build exists for this assembly."
}
```

---

## 7.2 `building`

Return `building` when:

- active run is still processing
- build stage not complete
- preview should be expected from the current run

Example triggers:

- assembly status = `running`
- selected run buildStatus = `queued` or `running`

---

## 7.3 `preparing`

Return `preparing` when:

- build succeeded
- preview session/publication is still being created
- preview URL is not yet ready

Typical causes:

- deployment/publish handoff still executing
- preview container still warming up
- preview record exists but no reachable URL yet

---

## 7.4 `ready`

Return `ready` when:

- preview target is valid
- preview URL is present
- preview has not expired
- source run is known

Set:

- `previewUrl`
- `entryUrl`
- `embeddable`
- `environment`
- `generatedAt`
- `updatedAt`

---

## 7.5 `failed`

Return `failed` when:

- build failed
- preview publication failed
- resolver can positively identify a failure state

Examples:

- buildStatus = `failed`
- deployment status = `failed`
- preview session creation error exists

This must be kept distinct from frontend iframe load failure. The current Preview placeholder incorrectly frames the tab as a document/artifact renderer, but the real Preview flow needs operational state separation.

assembly

---

## 7.6 `expired`

Return `expired` when:

- preview existed previously

- current preview target/session is no longer valid
- URL/session TTL elapsed

Conditions:

- `expiresAt < now`
- preview host reports expired session
- session/token lookup returns expired

---

# 8. Embeddable Rules

## `embeddable = true`

Use when:

- same-origin preview
- iframe-allowed host
- CSP/X-Frame-Options permit embedding
- internal preview container intended for embed

## `embeddable = false`

Use when:

- preview host blocks iframe rendering
- signed URL only meant for top-level navigation
- security policy forbids embed

Frontend behavior:

- show “cannot be embedded”
- prioritize “Open in New Tab”

---

# 9. Error Normalization Rules

Do not expose raw stack traces.

Map backend/internal failures to concise user-facing messages.

### Recommended normalized messages

- `"No preview-capable build exists for this assembly."`
- `"Build is still in progress."`
- `"Preview target is still being prepared."`
- `"The latest build failed and did not produce a preview."`
- `"The preview session expired."`
- `"The preview target exists but cannot be embedded."`
- `"Preview resolution failed."`

---

# 10. Resolver Output Examples

## 10.1 No build yet


```
{
 status: "none",
 assemblyId: 42,
 runId: null,
 buildStatus: null,
 previewUrl: null,
 entryUrl: null,
 updatedAt: "2026-03-11T10:15:00.000Z",
 source: "none",
 error: "No preview-capable build exists for this assembly."
}
```


## 10.2 Active build running


```
{
 status: "building",
 assemblyId: 42,
 runId: "run_20260311_001",
 buildStatus: "running",
 previewUrl: null,
 entryUrl: null,
 updatedAt: "2026-03-11T10:16:12.000Z",
 source: "active_run",
 error: "Build is still in progress."
}
```


## 10.3 Build finished, preview warming

{
 status: "preparing",
 assemblyId: 42,
 runId: "run_20260311_001",
 buildStatus: "completed",
 previewUrl: null,
 entryUrl: null,
 updatedAt: "2026-03-11T10:18:01.000Z",
 source: "active_run",
 error: "Preview target is still being prepared."
}

## 10.4 Ready and embeddable

{
 status: "ready",
 assemblyId: 42,
 runId: "run_20260311_001",
 buildStatus: "completed",
 previewUrl: "https://preview.axion.local/session/abc123",
 entryUrl: "/",
 updatedAt: "2026-03-11T10:19:15.000Z",
 generatedAt: "2026-03-11T10:19:10.000Z",
 expiresAt: "2026-03-11T12:19:10.000Z",
 embeddable: true,
 environment: "preview",
 source: "active_run",
 error: null
}

## 10.5 Build failed

{
 status: "failed",
 assemblyId: 42,
 runId: "run_20260311_001",
 buildStatus: "failed",
 previewUrl: null,
 entryUrl: null,
 updatedAt: "2026-03-11T10:17:44.000Z",
 source: "active_run",
 error: "The latest build failed and did not produce a preview."
}

## 10.6 Expired preview

```
{
 status: "expired",
 assemblyId: 42,
 runId: "run_20260310_008",
 buildStatus: "completed",
 previewUrl: null,
 entryUrl: null,
 updatedAt: "2026-03-11T10:21:00.000Z",
 generatedAt: "2026-03-10T20:10:10.000Z",
 expiresAt: "2026-03-10T22:10:10.000Z",
 embeddable: false,
 environment: "preview",
 source: "latest_ready_preview",
 error: "The preview session expired."
}
```

---

## 11. Route Handler Outline

```
app.get("/api/assemblies/:id/preview", async (req, res) => {
 const assemblyId = Number(req.params.id);

 const assembly = await getAssemblyById(assemblyId);
 if (!assembly) {
   return res.status(404).json({ error: "Assembly not found" });
 }

 const candidates = await getPreviewCandidatesForAssembly(assemblyId, assembly.runId);

 const resolved = await resolveAssemblyPreview({
   assembly,
   candidates,
   now: new Date(),
 });

 return res.json(resolved);
});
```

---

## 12. Resolver Function Breakdown

## 12.1 `getPreviewCandidatesForAssembly`

Returns ordered candidates such as:

- current run
- latest buildable runs
- latest ready preview-backed runs

Suggested shape:

```
type PreviewCandidate = {
 runId: string;
 runStatus: string | null;
 buildStatus: string | null;
 hasBuild: boolean;
 completedAt?: string | null;
 startedAt?: string | null;
 previewRecord?: {
   url: string | null;
   entryUrl?: string | null;
   embeddable?: boolean;
   environment?: string | null;
   generatedAt?: string | null;
   expiresAt?: string | null;
   status?: string | null;
   error?: string | null;
 } | null;
};
```

This fits the existing run/build orientation already visible in `BuildTabWithRuns`, where buildable runs carry `runId`, `status`, `hasBuild`, and `buildStatus`.

assembly

---

## 12.2 `resolveAssemblyPreview`

Input:

- assembly
- ordered candidates
- current time

Output:

- normalized `AssemblyPreviewResponse`

Pseudo-flow:

```
function resolveAssemblyPreview({ assembly, candidates, now }): AssemblyPreviewResponse {
 if (!candidates.length) return noneResponse(...);

 const active = pickActiveCandidate(candidates, assembly.runId);

 if (active && assembly.status === "running") {
   if (active.buildStatus === "failed") return failedResponse(...);
   if (active.previewRecord?.url) return readyOrExpired(active, now, "active_run");
   if (!active.hasBuild || active.buildStatus === "running") return buildingResponse(...);
   return preparingResponse(...);
 }

 const latestBuildable = pickLatestBuildableCandidate(candidates);
 if (latestBuildable) {
   if (latestBuildable.buildStatus === "failed") return failedResponse(...);
   if (latestBuildable.previewRecord?.url) return readyOrExpired(latestBuildable, now,
"latest_buildable_run");
   if (latestBuildable.hasBuild) return preparingResponse(...);
 }

 const latestReady = pickLatestReadyPreviewCandidate(candidates, now);
 if (latestReady) return readyOrExpired(latestReady, now, "latest_ready_preview");

 return noneResponse(...);
}
```

---

# 13. Optional Supporting Endpoints

## 13.1 Force re-resolution

`POST /api/assemblies/:id/preview/refresh`

Purpose:

- renew session
- re-run preview resolution

- repair stale state

Response:

- same shape as `GET /preview`

## 13.2 Direct run preview lookup

`GET /api/assemblies/:id/runs/:runId/preview`

Purpose:

- support future run switching in Preview tab
- inspect older build previews

## 13.3 Route manifest

`GET /api/assemblies/:id/preview/routes`

Purpose:

- support future route switcher in Preview toolbar

---

# 14. Backend Done Definition

This backend layer is complete when:

- Preview has a dedicated API source of truth
- the endpoint returns normalized lifecycle states
- the resolver chooses the correct source run
- build failure and preview expiry are distinct
- embeddable vs non-embeddable is explicit
- the frontend no longer has to infer preview readiness from files

# Assembly Preview — Server Implementation Outline

# Server Implementation Outline — Assembly Preview

## 1. Route Placement

### Recommended location

Put the route in the same server module that already owns:

- assembly read/update/delete
- assembly run control
- stage/gate detail reads

### Route family

Use:

- `GET /api/assemblies/:id/preview`
- optional later: `POST /api/assemblies/:id/preview/refresh`
- optional later: `GET /api/assemblies/:id/runs/:runId/preview`

That fits the current frontend query pattern, which already keys assembly data as `["/api/assemblies", id]` and calls `apiRequest("/api/assemblies/${id}")`.

assembly

---

## 2. Backend Ownership Model

The preview resolver should sit between three existing concepts:

### Assembly layer

Owns:

- assembly id
- current run id
- overall assembly status

The page already uses `assembly.runId`, `assembly.status`, `assembly.currentStep`, and `assembly.runs`, so the preview endpoint should be derived from those same records rather than inventing a separate lifecycle store first.

### Run/build layer

Owns:

- run ids
- run status
- whether build output exists
- build-specific status

This aligns with the existing build tab model, which is already treated as a run-aware companion to the assembly page.

assembly

### Preview publication layer

Owns:

- preview URL
- embeddable flag
- environment
- session expiry
- publication/preparation state

This may be new if you do not already persist preview targets.

---

# 3. Module Breakdown

## 3.1 Route Handler

Create a route handler whose only job is:

- validate input
- load assembly
- call preview resolver
- return normalized JSON

### Example shape

```
app.get("/api/assemblies/:id/preview", async (req, res) => {
 const assemblyId = Number(req.params.id);

 if (!Number.isFinite(assemblyId)) {
   return res.status(400).json({ error: "Invalid assembly id" });
 }

 const assembly = await storage.getAssemblyById(assemblyId);
 if (!assembly) {
   return res.status(404).json({ error: "Assembly not found" });
 }

 const preview = await resolveAssemblyPreview({ assemblyId, assembly });
 return res.json(preview);
});
```

---

## 3.2 Preview Resolver Service

Create a dedicated server-side function like:

```
async function resolveAssemblyPreview(args: {
 assemblyId: number;
 assembly: AssemblyRecord;
}): Promise<AssemblyPreviewResponse>
```

This function should contain the real logic for:

- candidate discovery
- status derivation
- preview source selection
- response normalization

Keep the route thin and the resolver thick.

---

## 3.3 Preview Repository / Access Layer

Add a small helper layer responsible for reading preview-capable data.

Suggested functions:

async function getAssemblyPreviewCandidates(assemblyId: number, currentRunId?: string | null)
async function getPreviewRecordForRun(runId: string)
async function getLatestBuildableRunsForAssembly(assemblyId: number)
async function getActiveRunForAssembly(assemblyId: number)

This keeps preview logic from being scattered across route handlers.

---

# 4. Data Sources the Resolver Needs

## 4.1 Assembly record

Required fields:

- `id`
- `runId`
- `status`
- optionally `currentStep`

These are already surfaced on the page.

## 4.2 Assembly runs

Required fields:

- `runId`
- `status`
- stage/build metadata if available
- timestamps if available

The page already treats `assembly.runs` as part of the assembly response and derives `latestRun` from them.

assembly

## 4.3 Build metadata

Required fields:

- `hasBuild`

- `buildStatus`
- build completion timestamp
- maybe build manifest location

This is implied by the existing build-tab separation, even if the exact backend schema is outside the uploaded file.

assembly

## 4.4 Preview publication/session metadata

Required fields:

- `previewUrl`
- `entryUrl`
- `environment`
- `embeddable`
- `generatedAt`
- `expiresAt`
- publication `status`
- failure reason

If you do not already have this table/object, this is the one new backing structure Preview likely needs.

---

# 5. Suggested Internal Types

```
type PreviewCandidate = {
 runId: string;
 runStatus: string | null;
 buildStatus: string | null;
 hasBuild: boolean;
 startedAt?: string | null;
 completedAt?: string | null;
 previewRecord?: PreviewRecord | null;
};

type PreviewRecord = {
 status: "preparing" | "ready" | "failed" | "expired";
 previewUrl: string | null;
 entryUrl: string | null;
```

```
 environment: string | null;
 embeddable: boolean;
 generatedAt?: string | null;
 expiresAt?: string | null;
 error?: string | null;
};

type AssemblyPreviewResponse = {
 status: "none" | "building" | "preparing" | "ready" | "failed" | "expired";
 assemblyId: number;
 runId: string | null;
 buildStatus: string | null;
 previewUrl: string | null;
 entryUrl: string | null;
 updatedAt: string | null;
 generatedAt?: string | null;
 expiresAt?: string | null;
 embeddable?: boolean;
 environment?: string | null;
 source?: "active_run" | "latest_buildable_run" | "latest_ready_preview" | "none";
 error?: string | null;
};
```

---

# 6. Resolver Pipeline

# Step 1 — Load assembly context

Start with:

- assembly record
- current run id
- current assembly status
- recent runs

Because the page already polls `/api/assemblies/:id` while `status === "running"`, Preview should derive its active behavior from the same lifecycle.

assembly

# Step 2 — Build ordered run candidates

Create candidates in this order:

1. current assembly run
2. latest buildable runs
3. latest run with valid preview record

## Step 3 — Resolve active state

Apply rules:

### If assembly is actively running

- if build not done: `building`
- if build done, preview not published: `preparing`
- if preview ready: `ready`
- if build failed: `failed`

### If assembly is not actively running

Prefer latest good build:

- ready preview → `ready`
- expired preview → `expired`
- failed build → `failed`
- no buildable run → `none`

## Step 4 — Normalize response

Always return a complete response shape, even if many fields are null.

---

# 7. Helper Function Set

# 7.1 Candidate assembly

```
async function buildPreviewCandidates(params: {
 assembly: AssemblyRecord;
}): Promise<PreviewCandidate[]>
```

Responsibilities:

- read assembly current run
- read latest runs

- join build state
- join preview record if present
- sort candidates

---

## 7.2 Active-run resolver

```
function resolveFromActiveRun(
 assembly: AssemblyRecord,
 candidate: PreviewCandidate,
 now: Date
): AssemblyPreviewResponse | null
```

Use this when:

- `assembly.status === "running"`
- candidate matches `assembly.runId`

---

## 7.3 Latest-build resolver

```
function resolveFromLatestBuildableRun(
 candidate: PreviewCandidate,
 now: Date
): AssemblyPreviewResponse | null
```

Use this when:

- assembly is not running
- a recent buildable run exists

---

## 7.4 Ready-preview resolver

```
function resolveReadyOrExpired(
 candidate: PreviewCandidate,
 now: Date,
 source: AssemblyPreviewResponse["source"]
): AssemblyPreviewResponse
```

This centralizes:

- TTL checks
- preview URL presence
- environment / embeddable mapping

---

## 7.5 Fallback responder

function resolveNoPreview(assemblyId: number, now: Date): AssemblyPreviewResponse

---

## 8. Persistence Options for Preview Metadata

## Option A — Store preview metadata per run

Best long-term option.

Create something like:

- `run_previews`
- or `assembly_run_preview_sessions`

Suggested columns:

- `id`
- `run_id`
- `status`
- `preview_url`
- `entry_url`
- `environment`
- `embeddable`
- `generated_at`
- `expires_at`
- `error`
- `updated_at`

### Why this is strongest

Preview is inherently run-scoped, and the rest of the page is already run-aware.

## Option B — Derive from build manifest on demand

Only use if build outputs already contain enough preview info.

Manifest would need fields like:

- preview URL
- route root
- publication status
- embed policy
- expiry

### Downside

Harder to distinguish `preparing` vs `failed` cleanly unless manifest generation is very disciplined.

## Option C — Resolve through a preview runtime service

If builds are launched into ephemeral sandboxes/containers, keep a service that maps:

- run id → preview session

### Good for

- short-lived preview sessions
- remote runtime previews
- session renewal flows

---

## 9. Suggested Route File Additions

If your server is organized with one assembly routes module, add:

### Read endpoint

router.get("/api/assemblies/:id/preview", getAssemblyPreviewHandler);

### Optional refresh endpoint

router.post("/api/assemblies/:id/preview/refresh", refreshAssemblyPreviewHandler);

### Optional historical endpoint

router.get("/api/assemblies/:id/runs/:runId/preview", getRunPreviewHandler);

Keep all three in the same assembly route family.

---

# 10. Refresh Endpoint Behavior

`POST /api/assemblies/:id/preview/refresh`

Use this only if preview resolution might need active regeneration.

Possible behavior:

1. load latest preview-capable run
2. if preview is expired, renew session
3. if preview is missing but build is complete, reattempt publish/register
4. return the same normalized preview response shape

This lets the frontend keep a clean mutation path later without inventing different response handling.

---

# 11. Error and Status Mapping

# Backend internal → frontend normalized

### Build still running

Internal:

- run status `running`
- build stage incomplete

Return:

- `status: "building"`

### Build succeeded, no preview target yet

Internal:

- build complete
- preview record absent or pending

Return:

- `status: "preparing"`

### Build failed

Internal:

- build status `failed`

Return:

- `status: "failed"`

### Preview session expired

Internal:

- expiry passed
- session invalidated

Return:

- `status: "expired"`

### No preview-capable source

Internal:

- no buildable runs
- no preview record
- no active build

Return:

- `status: "none"`

This separation matters because the frontend Preview tab should distinguish build failure from frame/embedding failure. The current placeholder text does not do that yet.

assembly

---

## 12. Security / Validity Checks

## 12.1 Validate assembly ownership/access

Before returning preview URLs, enforce the same access controls as the rest of the assembly routes.

## 12.2 Avoid exposing internal-only hosts

If preview URLs are internal service addresses, translate them into browser-safe public URLs first.

## 12.3 Strip raw internal errors

Do not expose stack traces or container IDs in `error`.

## 12.4 Validate expiry before returning `ready`

If expired, return `expired`, not stale `ready`.

---

# 13. Logging / Observability

Add structured logs for:

- preview resolution request
- chosen source run
- normalized status returned
- refresh attempts
- session renewal failures

Recommended fields:

- `assemblyId`
- `runId`
- `source`
- `status`
- `buildStatus`
- `embeddable`
- `expired`

This will make Preview debugging much easier once the tab goes live.

---

# 14. Testing Matrix

## Unit tests for resolver

Cover:

- no runs
- active build running
- build complete but preview pending
- ready preview
- failed build
- expired preview
- non-embeddable preview
- stale current run but valid prior preview

## Route tests

Cover:

- invalid assembly id
- assembly not found
- happy path ready preview
- refresh path if implemented

## Contract tests

Ensure the endpoint always returns the full normalized shape.

---

# 15. Suggested Implementation Sequence

### Pass 1

- add response type
- add `GET /api/assemblies/:id/preview`
- build basic resolver using assembly + runs

### Pass 2

- add preview metadata source
- return `ready / building / failed / none`

### Pass 3

- add `preparing` and `expired`
- add embeddable/environment fields

### Pass 4

- add refresh endpoint
- add logging
- add resolver tests

---

## 16. Practical Recommendation

The cleanest first implementation is:

- keep Preview under assembly routes
- make preview run-scoped
- store preview metadata on or beside runs
- normalize everything through one resolver
- let the frontend only consume one contract

That fits the structure your page already shows: assembly-level query, run-aware actions, a separate build tab, and a dedicated preview tab waiting to be turned into a real runtime surface.

# Assembly Preview — Schema + Helper Pack

# Assembly Preview — Schema + Helper Pack

## 1. Storage Model

### 1.1 Core rule

A preview belongs to a **run**.

Not:

- one preview per assembly forever
- one preview per tab session only
- one preview inferred from files

This is the right fit because Build is already run-based with `runId`, `hasBuild`, and `buildStatus`.

assembly

---

## 2. Recommended Table

### 2.1 `assembly_run_previews`

This is the main table.

#### Purpose

Stores the latest known preview publication/session state for a given run.

#### Columns

```
id              string   // pk
assembly_id     number   // fk -> assemblies.id
run_id          string   // unique-ish per run
status          string   // preparing | ready | failed | expired
build_status    string?  // queued | running | completed | failed
```

```
preview_url       string?  // browser-safe preview URL
entry_url         string?  // default route/path
environment       string?  // local | preview | staging | sandbox
embeddable        boolean  // iframe-safe or not
generated_at      datetime?
expires_at        datetime?
last_checked_at   datetime?
updated_at        datetime
created_at        datetime
error             text?
meta_json         json?    // optional extra runtime/session metadata
```

---

## 2.2 Constraints

### Minimum constraints

- index on `assembly_id`
- index on `run_id`
- composite index on `(assembly_id, updated_at desc)`
- optional unique on `run_id`

### Validation rules

- `status` must be one of:
  - `preparing`
  - `ready`
  - `failed`
  - `expired`
- `preview_url` required when `status = ready`
- `expires_at` optional, but if present must be later than `generated_at`

---

## 3. Why run-scoped storage is best

### It matches the current page

The page already thinks in:

- assembly

- current run
- buildable runs
- latest run

Not in terms of a detached preview object.

### It supports future features

- switch preview by run
- compare two previews
- keep preview history
- expire only one run's preview
- retry publication for a specific run

---

# 4. Optional Secondary Table

## 4.1 `assembly_run_preview_events`

Only add this if you want traceability.

### Purpose

Tracks preview lifecycle transitions.

### Columns

```
id            string
assembly_id   number
run_id        string
event_type    string   // created | preparing | ready | failed | expired | refreshed
from_status   string?
to_status     string?
message       text?
created_at    datetime
payload_json  json?
```

### Why add it

Useful if Preview becomes operationally important and you want an audit trail.

---

# 5. Suggested ORM Type Shape

```
type AssemblyRunPreviewRecord = {
 id: string;
 assemblyId: number;
 runId: string;
 status: "preparing" | "ready" | "failed" | "expired";
 buildStatus: string | null;
 previewUrl: string | null;
 entryUrl: string | null;
 environment: string | null;
 embeddable: boolean;
 generatedAt: string | null;
 expiresAt: string | null;
 lastCheckedAt: string | null;
 updatedAt: string;
 createdAt: string;
 error: string | null;
 metaJson: Record<string, any> | null;
};
```

---

# 6. Helper Function List

## 6.1 Read helpers

These are the minimum read helpers.

```
async function getAssemblyById(assemblyId: number)
async function getAssemblyRuns(assemblyId: number)
async function getBuildableRunsForAssembly(assemblyId: number)
async function getRunPreviewRecord(runId: string)
async function getLatestPreviewRecordForAssembly(assemblyId: number)
async function getPreviewRecordsForAssembly(assemblyId: number)
```

### What each does

- `getAssemblyById` → current assembly context
- `getAssemblyRuns` → all recent runs
- `getBuildableRunsForAssembly` → same run family Build tab already uses
- `getRunPreviewRecord` → preview state for a specific run
- `getLatestPreviewRecordForAssembly` → fallback/freshest preview

- `getPreviewRecordsForAssembly` → future history/compare support

---

## 6.2 Resolver helpers

These keep the route handler thin.

```
async function buildPreviewCandidates(args: {
 assemblyId: number;
 currentRunId: string | null;
})

function pickActiveRunCandidate(candidates: PreviewCandidate[], currentRunId: string | null)
function pickLatestBuildableCandidate(candidates: PreviewCandidate[])
function pickLatestReadyPreviewCandidate(candidates: PreviewCandidate[], now: Date)
function isPreviewExpired(record: AssemblyRunPreviewRecord, now: Date): boolean
function normalizePreviewResponse(args: {
 assemblyId: number;
 source: "active_run" | "latest_buildable_run" | "latest_ready_preview" | "none";
 candidate?: PreviewCandidate | null;
 now: Date;
})
```

---

## 6.3 Write/update helpers

These are needed once build/publication starts feeding Preview.

```
async function createRunPreviewRecord(input: CreateRunPreviewRecordInput)
async function updateRunPreviewStatus(runId: string, patch:
Partial<AssemblyRunPreviewRecord>)
async function markRunPreviewPreparing(runId: string, patch?:
Partial<AssemblyRunPreviewRecord>)
async function markRunPreviewReady(runId: string, patch: {
 previewUrl: string;
 entryUrl?: string | null;
 environment?: string | null;
 embeddable?: boolean;
 generatedAt?: string;
 expiresAt?: string | null;
 buildStatus?: string | null;
})
```

```
async function markRunPreviewFailed(runId: string, error: string, buildStatus?: string | null)
async function markRunPreviewExpired(runId: string)
async function touchRunPreviewChecked(runId: string, checkedAt?: string)
```

---

## 6.4 Optional event helpers

If you add the event table:

```
async function createRunPreviewEvent(input: {
 assemblyId: number;
 runId: string;
 eventType: string;
 fromStatus?: string | null;
 toStatus?: string | null;
 message?: string | null;
 payloadJson?: Record<string, any> | null;
})
```

---

## 7. Candidate Type

```
type PreviewCandidate = {
 runId: string;
 runStatus: string | null;
 buildStatus: string | null;
 hasBuild: boolean;
 startedAt?: string | null;
 completedAt?: string | null;
 previewRecord?: AssemblyRunPreviewRecord | null;
};
```

This matches the current build-tab style closely enough because Build already works from a run list with `runId`, `status`, `hasBuild`, and `buildStatus`.

assembly

---

## 8. Create/Update Lifecycle

## 8.1 When to create a preview row

Create a preview row when:

- a buildable run starts entering preview-capable stages
- or when a build finishes and preview publication begins

### Recommended timing

As soon as preview work is expected, insert:

- `status = preparing`

That gives the frontend a real `preparing` state instead of guessing.

---

## 8.2 When to mark `ready`

Mark `ready` when:

- preview URL is known
- target/session is usable
- expiry is known if applicable

Set:

- `preview_url`
- `entry_url`
- `environment`
- `embeddable`
- `generated_at`
- `expires_at`
- `last_checked_at`

---

## 8.3 When to mark `failed`

Mark `failed` when:

- build failed before preview publication
- preview publication/session creation failed
- preview host returned terminal failure

Set:

- `error`
- `build_status` if known

---

## 8.4 When to mark `expired`

Mark `expired` when:

- TTL has elapsed
- session refresh failed
- runtime target was explicitly torn down

---

# 9. Resolver Assembly Flow

## 9.1 `buildPreviewCandidates`

This function should:

1. load current assembly run
2. load buildable runs
3. join preview rows by run id
4. sort newest-first, with current run first if present

### Output

A normalized candidate list the resolver can consume without further DB work.

---

## 9.2 `normalizePreviewResponse`

This function should take:

- assembly
- candidate
- source
- current time

And always return:

- full `AssemblyPreviewResponse`
- no partial ad hoc objects

---

# 10. Suggested input/output types

## 10.1 Create input

```
type CreateRunPreviewRecordInput = {
 assemblyId: number;
 runId: string;
 status?: "preparing" | "ready" | "failed" | "expired";
 buildStatus?: string | null;
 previewUrl?: string | null;
 entryUrl?: string | null;
 environment?: string | null;
 embeddable?: boolean;
 generatedAt?: string | null;
 expiresAt?: string | null;
 error?: string | null;
 metaJson?: Record<string, any> | null;
};
```

## 10.2 API output

```
type AssemblyPreviewResponse = {
 status: "none" | "building" | "preparing" | "ready" | "failed" | "expired";
 assemblyId: number;
 runId: string | null;
 buildStatus: string | null;
 previewUrl: string | null;
 entryUrl: string | null;
 updatedAt: string | null;
 generatedAt?: string | null;
 expiresAt?: string | null;
 embeddable?: boolean;
 environment?: string | null;
 source?: "active_run" | "latest_buildable_run" | "latest_ready_preview" | "none";
 error?: string | null;
};
```

---

## 11. Minimal first-pass implementation

If you want the fastest path to working Preview:

### Phase 1

Add only:

- `assembly_run_previews`
- `getRunPreviewRecord`
- `createRunPreviewRecord`
- `markRunPreviewPreparing`
- `markRunPreviewReady`
- `markRunPreviewFailed`
- `resolveAssemblyPreview`

### Phase 2

Add:

- `expires_at`
- `markRunPreviewExpired`
- refresh logic
- event table

That is enough to support the frontend scaffold cleanly.

---

## 12. Done definition

This schema/helper layer is complete when:

- every preview state can be stored per run
- resolver can join run/build state with preview state
- Preview endpoint no longer needs guesswork
- future run switching and compare mode remain possible

# Drizzle-style table definition

# Drizzle-style table definition

## 1.1 Recommended table: `assembly_run_previews`

```
import {
 pgTable,
 text,
 timestamp,
 boolean,
 integer,
 jsonb,
 index,
 uniqueIndex,
} from "drizzle-orm/pg-core";

export const assemblyRunPreviews = pgTable(
 "assembly_run_previews",
 {
   id: text("id").primaryKey(),

   assemblyId: integer("assembly_id")
     .notNull(),

   runId: text("run_id")
     .notNull(),

   status: text("status")
     .$type<"preparing" | "ready" | "failed" | "expired">()
     .notNull()
     .default("preparing"),

   buildStatus: text("build_status")
     .$type<"queued" | "running" | "completed" | "failed" | null>(),

   previewUrl: text("preview_url"),

   entryUrl: text("entry_url"),

   environment: text("environment")
     .$type<"local" | "preview" | "staging" | "sandbox" | "production-like" | null>(),
```

```
  embeddable: boolean("embeddable")
    .notNull()
    .default(true),

  generatedAt: timestamp("generated_at", { withTimezone: true }),

  expiresAt: timestamp("expires_at", { withTimezone: true }),

  lastCheckedAt: timestamp("last_checked_at", { withTimezone: true }),

  updatedAt: timestamp("updated_at", { withTimezone: true })
    .notNull()
    .defaultNow(),

  createdAt: timestamp("created_at", { withTimezone: true })
    .notNull()
    .defaultNow(),

  error: text("error"),

  metaJson: jsonb("meta_json").$type<Record<string, any> | null>(),
 },
 (table) => ({
  assemblyIdx: index("assembly_run_previews_assembly_idx").on(table.assemblyId),
  runIdx: uniqueIndex("assembly_run_previews_run_uidx").on(table.runId),
  assemblyUpdatedIdx: index("assembly_run_previews_assembly_updated_idx").on(
    table.assemblyId,
    table.updatedAt,
  ),
  statusIdx: index("assembly_run_previews_status_idx").on(table.status),
 }),
);
```

## 1.2 Why this table shape

`runId` should be the core join key because the current Build tab is already run-based, and Preview should resolve against that same run/build surface rather than against raw artifact files or a single assembly-global preview record.

assembly

assembly

---

# 2. Optional event table

Use this only if you want lifecycle traceability.

```
import {
 pgTable,
 text,
 timestamp,
 integer,
 jsonb,
 index,
} from "drizzle-orm/pg-core";

export const assemblyRunPreviewEvents = pgTable(
 "assembly_run_preview_events",
 {
   id: text("id").primaryKey(),

   assemblyId: integer("assembly_id")
     .notNull(),

   runId: text("run_id")
     .notNull(),

   eventType: text("event_type")
     .$type<"created" | "preparing" | "ready" | "failed" | "expired" | "refreshed">()
     .notNull(),

   fromStatus: text("from_status")
     .$type<"preparing" | "ready" | "failed" | "expired" | null>(),

   toStatus: text("to_status")
     .$type<"preparing" | "ready" | "failed" | "expired" | null>(),

   message: text("message"),

   payloadJson: jsonb("payload_json").$type<Record<string, any> | null>(),

   createdAt: timestamp("created_at", { withTimezone: true })
     .notNull()
     .defaultNow(),
 },
 (table) => ({
```

```
  assemblyIdx: index("assembly_run_preview_events_assembly_idx").on(table.assemblyId),
  runIdx: index("assembly_run_preview_events_run_idx").on(table.runId),
  createdIdx: index("assembly_run_preview_events_created_idx").on(table.createdAt),
 }),
);
```

---

# 3. Type aliases for storage layer

```
export type PreviewRecordStatus = "preparing" | "ready" | "failed" | "expired";

export type PreviewBuildStatus = "queued" | "running" | "completed" | "failed" | null;

export type PreviewEnvironment =
 | "local"
 | "preview"
 | "staging"
 | "sandbox"
 | "production-like"
 | null;

export type AssemblyRunPreviewRecord = typeof assemblyRunPreviews.$inferSelect;
export type NewAssemblyRunPreviewRecord = typeof assemblyRunPreviews.$inferInsert;
```

---

# 4. Create/update input types

```
export type CreateRunPreviewRecordInput = {
 id: string;
 assemblyId: number;
 runId: string;
 status?: PreviewRecordStatus;
 buildStatus?: PreviewBuildStatus;
 previewUrl?: string | null;
 entryUrl?: string | null;
 environment?: PreviewEnvironment;
 embeddable?: boolean;
 generatedAt?: Date | null;
 expiresAt?: Date | null;
 lastCheckedAt?: Date | null;
 error?: string | null;
 metaJson?: Record<string, any> | null;
```

```
};

export type UpdateRunPreviewRecordPatch = Partial<{
 status: PreviewRecordStatus;
 buildStatus: PreviewBuildStatus;
 previewUrl: string | null;
 entryUrl: string | null;
 environment: PreviewEnvironment;
 embeddable: boolean;
 generatedAt: Date | null;
 expiresAt: Date | null;
 lastCheckedAt: Date | null;
 error: string | null;
 metaJson: Record<string, any> | null;
 updatedAt: Date;
}>;
```

---

# 5. Storage method signatures

## 5.1 Read methods

```
export interface PreviewStorage {
 getRunPreviewRecord(runId: string): Promise<AssemblyRunPreviewRecord | null>;

 getPreviewRecordsForAssembly(
   assemblyId: number,
 ): Promise<AssemblyRunPreviewRecord[]>;

 getLatestPreviewRecordForAssembly(
   assemblyId: number,
 ): Promise<AssemblyRunPreviewRecord | null>;
}
```

## 5.2 Write methods

```
export interface PreviewStorageMutations {
 createRunPreviewRecord(
   input: CreateRunPreviewRecordInput,
 ): Promise<AssemblyRunPreviewRecord>;

 updateRunPreviewRecord(
```

```
  runId: string,
  patch: UpdateRunPreviewRecordPatch,
): Promise<AssemblyRunPreviewRecord | null>;

markRunPreviewPreparing(
  runId: string,
  patch?: Partial<Pick<
    UpdateRunPreviewRecordPatch,
    "buildStatus" | "entryUrl" | "environment" | "metaJson"
  >>,
): Promise<AssemblyRunPreviewRecord | null>;

markRunPreviewReady(
  runId: string,
  patch: {
    previewUrl: string;
    entryUrl?: string | null;
    environment?: PreviewEnvironment;
    embeddable?: boolean;
    buildStatus?: PreviewBuildStatus;
    generatedAt?: Date | null;
    expiresAt?: Date | null;
    lastCheckedAt?: Date | null;
    metaJson?: Record<string, any> | null;
  },
): Promise<AssemblyRunPreviewRecord | null>;

markRunPreviewFailed(
  runId: string,
  error: string,
  buildStatus?: PreviewBuildStatus,
  metaJson?: Record<string, any> | null,
): Promise<AssemblyRunPreviewRecord | null>;

markRunPreviewExpired(
  runId: string,
  lastCheckedAt?: Date,
): Promise<AssemblyRunPreviewRecord | null>;

touchRunPreviewChecked(
  runId: string,
  checkedAt?: Date,
): Promise<AssemblyRunPreviewRecord | null>;
}
```

# 6. Resolver helper signatures

These are the helpers that sit above storage and below the route.

```
export type PreviewCandidate = {
 runId: string;
 runStatus: string | null;
 buildStatus: PreviewBuildStatus;
 hasBuild: boolean;
 startedAt?: string | null;
 completedAt?: string | null;
 previewRecord?: AssemblyRunPreviewRecord | null;
};

export async function buildPreviewCandidates(args: {
 assemblyId: number;
 currentRunId: string | null;
}): Promise<PreviewCandidate[]>;

export function pickActiveRunCandidate(
 candidates: PreviewCandidate[],
 currentRunId: string | null,
): PreviewCandidate | null;

export function pickLatestBuildableCandidate(
 candidates: PreviewCandidate[],
): PreviewCandidate | null;

export function pickLatestReadyPreviewCandidate(
 candidates: PreviewCandidate[],
 now: Date,
): PreviewCandidate | null;

export function isPreviewExpired(
 record: AssemblyRunPreviewRecord,
 now: Date,
): boolean;
```

The reason to keep these helpers run-centric is that the current page already anchors build behavior around buildable runs rather than around files or a detached preview object.

assembly

---

# 7. Route response types

```
export type PreviewResponseStatus =
 | "none"
 | "building"
 | "preparing"
 | "ready"
 | "failed"
 | "expired";

export type AssemblyPreviewResponse = {
 status: PreviewResponseStatus;
 assemblyId: number;
 runId: string | null;
 buildStatus: PreviewBuildStatus;
 previewUrl: string | null;
 entryUrl: string | null;
 updatedAt: string | null;
 generatedAt?: string | null;
 expiresAt?: string | null;
 embeddable?: boolean;
 environment?: PreviewEnvironment;
 source?: "active_run" | "latest_buildable_run" | "latest_ready_preview" | "none";
 error?: string | null;
};
```

---

# 8. Example Drizzle-backed storage implementation signatures

```
import { db } from "../db";
import { eq, desc } from "drizzle-orm";

export async function getRunPreviewRecord(
 runId: string,
): Promise<AssemblyRunPreviewRecord | null> {
 // implementation
```

```
}

export async function getPreviewRecordsForAssembly(
 assemblyId: number,
): Promise<AssemblyRunPreviewRecord[]> {
 // implementation
}

export async function getLatestPreviewRecordForAssembly(
 assemblyId: number,
): Promise<AssemblyRunPreviewRecord | null> {
 // implementation
}

export async function createRunPreviewRecord(
 input: CreateRunPreviewRecordInput,
): Promise<AssemblyRunPreviewRecord> {
 // implementation
}

export async function updateRunPreviewRecord(
 runId: string,
 patch: UpdateRunPreviewRecordPatch,
): Promise<AssemblyRunPreviewRecord | null> {
 // implementation
}
```

---

# 9. Example method bodies pattern

## 9.1 `getRunPreviewRecord`

```
export async function getRunPreviewRecord(runId: string) {
 const rows = await db
   .select()
   .from(assemblyRunPreviews)
   .where(eq(assemblyRunPreviews.runId, runId))
   .limit(1);

 return rows[0] ?? null;
}
```

## 9.2 `getLatestPreviewRecordForAssembly`

```
export async function getLatestPreviewRecordForAssembly(assemblyId: number) {
 const rows = await db
   .select()
   .from(assemblyRunPreviews)
   .where(eq(assemblyRunPreviews.assemblyId, assemblyId))
   .orderBy(desc(assemblyRunPreviews.updatedAt))
   .limit(1);

 return rows[0] ?? null;
}
```

## 9.3 `createRunPreviewRecord`

```
export async function createRunPreviewRecord(input: CreateRunPreviewRecordInput) {
 const rows = await db
   .insert(assemblyRunPreviews)
   .values({
     id: input.id,
     assemblyId: input.assemblyId,
     runId: input.runId,
     status: input.status ?? "preparing",
     buildStatus: input.buildStatus ?? null,
     previewUrl: input.previewUrl ?? null,
     entryUrl: input.entryUrl ?? null,
     environment: input.environment ?? null,
     embeddable: input.embeddable ?? true,
     generatedAt: input.generatedAt ?? null,
     expiresAt: input.expiresAt ?? null,
     lastCheckedAt: input.lastCheckedAt ?? null,
     error: input.error ?? null,
     metaJson: input.metaJson ?? null,
   })
   .returning();

 return rows[0];
}
```

---

# 10. Minimal migration outline

```
create table assembly_run_previews (
```

```
 id text primary key,
 assembly_id integer not null,
 run_id text not null,
 status text not null default 'preparing',
 build_status text,
 preview_url text,
 entry_url text,
 environment text,
 embeddable boolean not null default true,
 generated_at timestamptz,
 expires_at timestamptz,
 last_checked_at timestamptz,
 updated_at timestamptz not null default now(),
 created_at timestamptz not null default now(),
 error text,
 meta_json jsonb
);

create unique index assembly_run_previews_run_uidx
 on assembly_run_previews (run_id);

create index assembly_run_previews_assembly_idx
 on assembly_run_previews (assembly_id);

create index assembly_run_previews_assembly_updated_idx
 on assembly_run_previews (assembly_id, updated_at desc);

create index assembly_run_previews_status_idx
 on assembly_run_previews (status);
```

---

# 11. Practical recommendation

Start with just:

- `assembly_run_previews`
- `getRunPreviewRecord`
- `createRunPreviewRecord`
- `updateRunPreviewRecord`
- `markRunPreviewPreparing`
- `markRunPreviewReady`
- `markRunPreviewFailed`

- `markRunPreviewExpired`

That is enough to power the real Preview tab cleanly while keeping it aligned with the existing assembly/build/run model already present in `assembly.tsx`.

assembly

assembly